Recall or Relearn? A Mixture of In-Context Experts for Recurring Concept Drift

Anonymous Authors

Abstract—Tabular foundation models (TFMs) learn a data stream through their context, so a past concept can be recalled by putting its labelled rows back into the context, without retraining. We characterise when this memory improves prediction under recurring concept drift. When a stored context holds one concept and the experts are separated by a margin, the gain of recall over relearning is the area between the stored context's accuracy and the TFM's learning curve, up to a first-batch term. The characterisation predicts large gains for concepts that need many examples and little gain for concepts that are learned quickly. Under a posterior-predictive model of the TFM, for deterministic concepts with a shared input distribution and in the population limit, a stored context recalls the concept that holds the majority of its rows; stored contexts are segments that start at changes detected by a two-sample test on the window's errors, with false-alarm and detection guarantees; if every change is detected and declared within ν rows of its boundary, a segment holds at most 2ν rows of other concepts. We propose MICE, a mixture of window and stored-context experts under a meta expert with two time scales. The fast level identifies a recurring concept within one batch under a margin; the slow level bounds the discounted half-Brier regret against the plain window by $2\ln 2 \approx 1.39$ at every time, for any stream. A model-oriented discrepancy organises the stored contexts. On a controlled grid, the gains predicted from learning curves rank the observed gains with Spearman correlations of 0.94 and 0.83, and the measured area term matches its prediction within 0.7 points in every cell. With fixed 500-row segments the gain depended on where the concept boundaries fell inside the segments; on 72 fresh grid streams with random boundary phases, MICE exceeds reset detection by 2.85 points on average (0.68 with fixed segments), on 70 of the 72 streams and in each of three block-length regimes (hierarchical 95% lower bounds +1.22 to +4.59), and by 2.69 points on all of 72 further streams from unused seeds; its stored contexts were majority-pure at every segment close, and 88% held at least 95% rows of one concept, short of the registered 90%, so these gains are measured with contexts that are mixed, not with the pure contexts the recall theorem assumes. On 29 real streams MICE is more accurate than all four in-context baselines on 23, and its one-sided 95% block-bootstrap bound on the difference to the plain window is above −0.3 points on 28 (backbone seed 0, fixed segments; detected segments differ by 0.01 points on average). With fixed segments, against an unmerged archive and a snapshot pool whose size was chosen, by timing alone, to fit the time per batch of MICE on three real prefixes, MICE is more accurate on the aligned grid (by 6.7 and 7.5 points) and on average over nine real streams (by 0.35 and 0.27), and level with the snapshot pool on the shifted grid, as are both pools with four experts, which take more time per batch than MICE. With detected segments, MICE takes 1.07 s per batch on the three prefixes, against 2.58 s for an archive of twelve experts with the same segments; it is 5.7 to 6.6 points more accurate than a snapshot pool of two experts on fresh, shifted and misaligned grid streams, and 1.3 to 3.4 points more accurate than one of twelve, which takes 1.3 to 2.6 times as long, except on 2 000-row blocks, where the two are level.

Index Terms—Concept drift, recurring concepts, tabular foundation models, in-context learning, data streams, online learning, mixture of experts.

1 Introduction

A data stream revisits its past: seasons, weekdays, market regimes and sensor modes return. A learner that kept what it knew about a concept can use it again when the concept recurs [1, 2]. For a trained model this means keeping a pool of models and deciding which one to reuse [3–5]. Tabular foundation models (TFMs) such as TabPFN [6] and TabICL [7] change the cost of this idea. A TFM approximates posterior prediction in one forward pass given a context of labelled rows [8, 9]; its knowledge of a concept is a set of rows, and recalling the concept means conditioning on them. Memory needs no training.

TFMs have already replaced incremental trees as stream learners: with a sliding memory they outperform adaptive tree ensembles on non-stationary benchmarks [10], and a prior with drifting causal models improves them under temporal shift [11]. Neither line reuses past contexts without additional training. A sliding context forgets a concept once it leaves the window, and a drift-extrapolating prior models a trend rather than a return. Three observations show what an in-context stream learner is missing, and what a remedy has to respect.

Observation 1 (Sliding contexts forget). In the five batches after a concept recurs, a TFM with a first-in-first-out (FIFO) context or with the short- and long-term memory (LTM) of Lourenço et al. [10] reaches 0.17, 0.39 and 0.51 accuracy on Sine [12], STAGGER [13] and Agrawal [14] streams, whereas the same TFM conditioned on the past rows of the recurring concept reaches at least 0.99; in steady state all contexts are equally accurate.

Observation 2 (Relearning in context is cheap, for simple concepts). A TFM relearns the concepts of the standard drift generators within about 200 rows, so resetting the context when a drift detector fires [12] already closes most of the gap of Observation 1 (0.81–0.88 in the same five batches). Memory pays only for concepts that take many rows to learn: on mixtures with 100 centroids the TFM needs more than

1000 rows to approach its asymptote.

Observation 3 (Resetting is unsafe). On 29 real streams the same reset detector is on average 1.8 points below the plain FIFO context and up to 9.6 points below it: most rises of the error rate on real streams do not mean that the context has become useless.

These observations raise the question of this paper: when should an in-context learner recall a past concept rather than relearn it, and how can it do so without paying when there is nothing to recall?

Approach. We treat every past context as an offline expert and the current window as an online expert, and combine them with a meta expert, following the mixture of online and offline experts (MOOE) of Zhao et al. [5]. An in-context expert costs no training, which changes both the method and its analysis. (i) Which segments belong to the same concept is decided by a model-oriented discrepancy in the sense of R-divergence [15]; we show that the original form is blind to concepts that contradict each other on the same inputs, which is the defining case of recurring drift, and that an excess-risk form repairs it. (ii) In the population limit, a stored context of deterministic concepts with a shared input distribution identifies the concept that holds the majority of its rows, so segments start at changes detected by a two-sample test on the window's errors; if every change is detected and declared within ν rows of its boundary, a segment holds at most 2ν rows of other concepts, wherever the boundaries fall. (iii) The meta expert has two time scales. A fast level restarts every batch and identifies a recurring concept in one batch under a margin; a slow level decides whether following the fast level is worth it at all, with a worst-case guarantee against the plain window. (iv) The gain of recall over relearning then has a closed form in the TFM's learning curve, so the value of memory can be estimated before deployment from labelled data representative of the concepts.

Contributions. (1) The value of recall. The gain of recalling a concept rather than relearning it is the area between the stored context's accuracy and the backbone's learning curve, under single-concept contexts and identification conditions (Theorem 1); it explains when memory pays and when the opportunity vanishes. (2) Theory for identifying and combining contexts. Majority purity as the population condition under which a mixed context of deterministic concepts with a shared input distribution recalls a concept, false-alarm and detection guarantees for the change test, and a bound on the mixing of segments whose changes are detected and declared within ν rows of the boundary (Propositions 2 and 3 and Corollary 1); identification within one batch under a margin (Proposition 4); a blind spot of R-divergence under conflicting concepts and its excess-risk repair under posterior-predictive assumptions (Proposition 1); and an anytime discounted half-Brier regret bound for the slow level (Theorem 2). (3) A training-free streaming method. MICE combines window and stored-context experts through a fast and a slow level and organises a bounded pool of past contexts, segmented at detected changes, by directed recall regret. (4) Evidence in the recall and the weak-gain regime. On a controlled grid, predicted and observed gains have Spearman correlations of 0.94 and 0.83; fixed segments lose the gain when boundaries fall inside them, and detected segments recover it on 72 fresh streams with random boundary phases (+2.85 points over reset, with contexts that are majority-pure but not pure) and on 72 confirmation streams (+2.69); on real streams, accuracy is non-inferior to the default window on 28 of 29 streams at a 0.3-point margin. Pool controls, ablations and resource comparisons quantify the roles of aggregation, pool organisation and larger windows.

2 Recurring Concept Drift for In-Context Learners

Setting. A stream delivers batches of B rows; the labels of a batch arrive one batch late. A frozen TFM $f_\Theta(\cdot \mid \mathbf{x}; \mathcal{C})$ predicts every batch from a context $\mathcal{C}$ of at most M labelled rows, and a context policy decides which rows form the context. The stream is a sequence of intervals $I_1, I_2, \ldots$ with distributions $P_1, P_2, \ldots$ over $(\mathbf{x}, y)$; concept drift occurs between intervals in the sense of Lu et al. [2].

Definition 1 (Recurring in-context concept drift). Let $\mathcal{C}_k$ be a context of rows of interval I_k and $f_\Theta(\cdot\,; \mathcal{C}_k)$ the k-th in-context expert. The drift into interval I_{K+1} is recurring if $\min_{k \le K} \mathrm{D}_T(P_{K+1} \,|\, P_k) \le \delta$, where $\mathrm{D}_T(Q \,|\, P)$ is the risk on Q of the expert of P minus that of the expert of Q (Definition 2), and new otherwise.

The definition is model-oriented: a drift is recurring when some stored context serves the new interval as well as its own rows would. Three policies are the reference points of this paper. FIFO is the plain sliding window: the context is the last M labelled rows. It is the simplest context policy and the building block of LTM [10]. Reset empties the context when the drift detection method (DDM) of Gama et al. [12] fires. Window ensemble mixes FIFO contexts of several lengths. None of them has memory.

3 Theory: From a Model-Oriented Discrepancy to Recall versus Relearn

The analysis has five steps, one per component of the method: which contexts share a concept (Section 3.1), which rows of the stream form a concept (Section 3.2), how fast a recurring concept is identified (Section 3.3), what recalling it is worth (Section 3.4), and what following the mixture can cost when nothing recurs (Section 3.5). All proofs are in Appendix A.

Notation. For a concept P over $(\mathbf{x}, y)$ and a loss ℓ, the risk of a context $\mathcal{C}$ is

$$\epsilon_P(\mathcal{C}) = \mathbb{E}_{(\mathbf{x},y)\sim P}\, \ell\big(f_\Theta(\cdot \mid \mathbf{x}; \mathcal{C}), y\big),$$

and the learning curve of P is $R_P(n) = \mathbb{E}_{\mathcal{C}\sim P^n}\, \epsilon_P(\mathcal{C})$. We write $p_\mathbf{x} = P(\cdot \mid \mathbf{x})$ for the conditional, P_X for the marginal, H for the entropy and JS for the Jensen–Shannon divergence.

3.1 Which Contexts Share a Concept

R-divergence [15] declares two distributions identical for a model when the hypothesis learned on their mixture has the same risk on both. For an in-context learner that hypothesis is the TFM conditioned on the union of the two contexts:

$$\mathrm{D}_R(P \| Q) = \big|\epsilon_P(\mathcal{C}_P \cup \mathcal{C}_Q) - \epsilon_Q(\mathcal{C}_P \cup \mathcal{C}_Q)\big|. \qquad (1)$$

Its population behaviour follows from the principle behind prior-fitted networks, which approximate the posterior predictive distribution [8, 9].

Assumption 1 (Posterior-predictive limit). With a context from P alone, the TFM predicts the conditional $p_{\mathbf{x}}$; with contexts of equal size from P and Q, it predicts the conditional of the mixture $U = \frac{1}{2}(P+Q)$,

$$u_{\mathbf{x}} = \alpha(\mathbf{x})\, p_{\mathbf{x}} + \big(1-\alpha(\mathbf{x})\big)\, q_{\mathbf{x}}, \qquad \alpha = \frac{P_X}{P_X + Q_X}.$$

We denote the two predictors by h_P and h_U.

Definition 2 (Recall regret and excess-risk discrepancy). For two concepts P and Q, the recall regret of P on Q is the risk that the context of P incurs on Q beyond that of the context of Q itself, and the excess-risk discrepancy is the mean risk that pooling the two contexts adds to each concept:

$$\mathrm{D}_T(Q \,|\, P) = \epsilon_Q(h_P) - \epsilon_Q(h_Q), \tag{2}$$

$$\mathrm{D}_E(P,Q) = \tfrac{1}{2}\big[\epsilon_P(h_U) - \epsilon_P(h_P)\big] + \tfrac{1}{2}\big[\epsilon_Q(h_U) - \epsilon_Q(h_Q)\big]. \tag{3}$$

Proposition 1 (Blind spot of R-divergence and its repair). Let Assumption 1 hold and let ℓ be the log-loss.

(i) Decomposition. The excess-risk discrepancy is the part of the divergence between the joints that covariate shift does not explain:

$$\mathrm{D}_E(P,Q) = \mathrm{JS}(P,Q) - \mathrm{JS}(P_X, Q_X) \;\geq\; 0.$$

If $P_X = Q_X$, then $\mathrm{D}_E(P,Q) = \mathbb{E}_{\mathbf{x}}\,\mathrm{JS}(p_{\mathbf{x}}, q_{\mathbf{x}})$, which is zero if and only if $p_{\mathbf{x}} = q_{\mathbf{x}}$ almost surely.

(ii) Blind spot. Let $P_X = Q_X$ and let both concepts be deterministic, $y = g_P(\mathbf{x})$ and $y = g_Q(\mathbf{x})$, with disagreement mass $\rho = \Pr_{\mathbf{x}}[g_P(\mathbf{x}) \neq g_Q(\mathbf{x})]$. Then, for every $\rho \in [0,1]$,

$$\mathrm{D}_R(P\|Q) = 0 \qquad \text{whereas} \qquad \mathrm{D}_E(P,Q) = \rho \ln 2.$$

(iii) 0-1 loss. In the setting of (ii) with the 0-1 loss, $\mathrm{D}_T(Q \,|\, P) = \rho$ for any rule that breaks ties of h_U. If ties are broken uniformly at random between the two maximisers, independently of the source of the query, and each risk is averaged over this randomisation before the difference is taken, then $\mathrm{D}_R(P\|Q) = \big|\mathbb{E}_\xi \epsilon_P(h_U^\xi) - \mathbb{E}_\xi \epsilon_Q(h_U^\xi)\big| = 0$. Under a deterministic tie rule $c(\mathbf{x})$, $\mathrm{D}_R(P\|Q) = \big|\mathbb{E}_{\mathbf{x}} \mathbf{1}[g_P(\mathbf{x}) \neq g_Q(\mathbf{x})]\big(\mathbf{1}[c(\mathbf{x}) \neq g_P(\mathbf{x})] - \mathbf{1}[c(\mathbf{x}) \neq g_Q(\mathbf{x})]\big)\big| \leq \rho$, which depends on both the disagreement region and the tie choices, is not determined by ρ alone, and can vanish although $\rho > 0$.

Remark 1. R-divergence asks whether the mixture hypothesis treats the two sets equally. Under the log-loss, two concepts that contradict each other on the same inputs are treated equally badly, so the answer is yes although neither context helps the other (Proposition 1(ii)); this is the defining case of recurring drift. Under the 0-1 loss the answer depends on how the mixture breaks its ties: it is again zero under uniform, source-independent random tie-breaking with risks averaged before taking their difference, and otherwise it is not determined by the disagreement mass ρ (Proposition 1(iii)). Under the 0-1 loss the directed form D_T equals ρ whatever the tie rule; under the log-loss it is infinite on the disagreement region of two deterministic concepts. The excess-risk form asks whether pooling costs anything. With finite contexts it has a second term: if $P = Q$, pooling doubles the context and changes the risk by $R_P(2n) - R_P(n) \leq 0$. The empirical excess is interference minus pooling gain, so a rule that pools two contexts when the empirical excess is at most δ does so when the pooling gain is at least the interference minus δ ($\delta = 0$ gives the exact equivalence). MICE instead merges on the directed form (2), which bounds the regret that recalling the stored expert would incur on the segment (Definition 1).

3.2 Which Rows Form a Concept

Theorem 1 below compares relearning with the stored expert of concept k. A stored context is built from segments of the stream, and a segment that straddles a concept boundary holds rows of two concepts. The next results say how much mixing identification tolerates, how reliably a change detector declares changes, and how much mixing remains when its declarations lie close to the boundaries.

Proposition 2 (Majority purity). Let Assumption 1 extend to unequal shares: a context whose rows come from P in proportion π and from Q in proportion $1-\pi$ predicts the conditional of $\pi P + (1-\pi)Q$. Let $P_X = Q_X$, let both concepts be deterministic, $y = g_P(\mathbf{x})$ and $y = g_Q(\mathbf{x})$, and let ℓ be the 0-1 loss. Then the recall regret of this context on P is

$$\mathrm{D}_T\big(P \,|\, \pi P + (1-\pi)Q\big) = \begin{cases} 0, & \pi > \frac{1}{2}, \\ \rho, & \pi < \frac{1}{2}, \end{cases}$$

where $\rho = \Pr_{\mathbf{x}}[g_P(\mathbf{x}) \neq g_Q(\mathbf{x})]$; at $\pi = \frac{1}{2}$ it depends on the tie rule, as in Proposition 1(iii).

Remark 2. In the population limit a stored context of deterministic concepts with a shared input distribution recalls the concept that holds the majority of its rows, so identifying the concept needs majority purity, not purity. For labels that are not deterministic a majority is not enough: if the most probable label of P at $\mathbf{x}$ exceeds every other label by $m(\mathbf{x})$, the mixture keeps it whenever $\pi > 1/(1+m(\mathbf{x}))$, since the part from Q changes each difference by at most $1-\pi$; deterministic concepts have $m = 1$. With finite contexts the minority rows are wrong labels for the majority concept on the disagreement region and correct ones elsewhere, and the proposition does not say how many clean rows a mixed context is worth; Theorem 1 is stated for pure contexts. Dropping the rows near a boundary raises π but lowers the number of stored rows.

Change detection. Let $z_t \in \{0, \ldots, B\}$ be the number of errors that the window of the last 300 rows makes on batch t, scored when its labels arrive. At time t, for every candidate change $b \in \{t - H_{\mathrm{cp}}, \ldots, t-1\}$, compare the error rate on the batches $b, \ldots, t-1$ (n_2 rows) with that on up to R_{cp} batches before b (n_1 rows), and declare a change at the candidate with the largest excess

$$\frac{1}{n_2}\sum_{s=b}^{t-1} z_s - \frac{1}{n_1}\sum_{s=b-R_{\mathrm{cp}}}^{b-1} z_s - \tau(n_1, n_2), \tag{4}$$

if that excess is positive, where

$$\tau(n_1, n_2) = \sqrt{\tfrac{1}{2}\big(\tfrac{1}{n_1} + \tfrac{1}{n_2}\big)\ln\tfrac{1}{\delta_{\mathrm{cp}}}}.$$

The compared batches before b start at or after the batch that follows the previous declared change, and at least two are required; n_1 and n_2 count B rows per compared batch.

Assumption 2 (Window error). Given all earlier rows and errors, the window errs on the rows of a batch independently, each with the conditional error rate of that batch, and almost surely: (a) between two changes this rate does not increase from batch to batch; (b) after a change at batch b, the mean rate on the batches $b, \dots, t-1$ exceeds that on every run of two to R_{cp} consecutive batches that ends at batch $b-1$ and starts at or after the batch following the previous declared change by at least β.

Proposition 3 (Detection). Under Assumption 2:

(i) False alarms. If no change occurs in the batches $t - H_{\mathrm{cp}} - R_{\mathrm{cp}}, \dots, t-1$, the probability that (4) declares one at t is at most $(R_{\mathrm{cp}} - 1)H_{\mathrm{cp}}\delta_{\mathrm{cp}}$.

(ii) Detection. If a change occurs at batch b and $\beta \geq 2\tau(2B, n_2)$ at time t, the probability that b is a candidate at t with at least two compared batches before it and that its excess at t is not positive is at most $(R_{\mathrm{cp}} - 1)\delta_{\mathrm{cp}}$. If b is such a candidate almost surely, a change is declared at t with probability at least $1 - (R_{\mathrm{cp}} - 1)\delta_{\mathrm{cp}}$.

Corollary 1 (Segment purity). If every declared change lies within ν rows of a concept boundary and no boundary is missed, then a closed segment S holds at most 2ν rows of other concepts, and it is majority-pure when $4\nu < |S|$.

Remark 3. With $B = 100$, $H_{\mathrm{cp}} = 3$, $R_{\mathrm{cp}} = 5$ and $\delta_{\mathrm{cp}} = 10^{-3}$, the false-alarm probability per batch is at most 0.012. By Proposition 2, the window, whose 300 rows hold the previous concept in the majority for two batches after a change, keeps predicting that concept on those batches; for two deterministic concepts that disagree on mass ρ, the gap β is about ρ on batches b and $b+1$ when the window erred little on the previous concept. With $n_2 = 200$ and the shortest reference, $n_1 = 200$, $2\tau = 0.372$. For $\beta \geq 0.372$, the probability that b is a candidate two batches later with at least two compared batches before it and that its excess is not positive is at most 0.004; if b is such a candidate almost surely, a change is declared then with probability at least 0.996. Under the conditions above, $\beta \geq 0.372$ holds for concepts that disagree on more than about 37% of the inputs. Changes between closer concepts may be missed, and a segment then holds both. When the declared batch is the one that holds the boundary or the next, $\nu < B$, and a segment of at least 400 rows is majority-pure. Under this location condition, which Proposition 3 does not guarantee, these results bound the mixing of a detected segment but do not make it pure, and Theorem 1 is stated for pure contexts. Detection, the location of each declared change and the purity of the stored contexts are separate conditions; Section 5.2 measures each.

3.3 The Fast Level: Identification under a Margin

Following Zhao et al. [5], the regret of a mixture of experts against the best predictor splits into the regret of the meta expert against the best expert and the regret of that expert,

$$\mathrm{Regret} = \mathrm{Regret}_{\mathrm{ME}} + \mathrm{Regret}_{\mathrm{KE}}. \tag{5}$$

MOOE bounds $\mathrm{Regret}_{\mathrm{ME}} \leq \sqrt{T \ln K}$ for cumulative losses over an interval of T rounds. After a recurrence the cumulative loss still carries the previous concept, so the fast level of MICE restarts every batch: it weights the K experts by

$$w_k \propto \exp(-\eta\, \ell_k), \tag{6}$$

where ℓ_k is the log-loss of expert k on the last labelled batch only. Its guarantee comes from a margin.

Assumption 3 (Margin). One expert $k^\star$ leads the last labelled batch by a margin $g > 0$: $\ell_{k^\star} \leq \ell_k - g$ for all $k \neq k^\star$.

Lemma 1 (Margin). Under Assumption 3, let $\kappa = \ln\bigl(1+(K-1)e^{-\eta g}\bigr)$. The mixture with the weights (6) satisfies:

(i) the leader holds almost all weight, $w_{k^\star} \geq e^{-\kappa}$;

(ii) on every row, the log-loss of the mixture is at most that of expert $k^\star$ plus κ;

(iii) the mixture predicts the class of expert $k^\star$ on every row where the top two probabilities of that expert differ by more than $e^\kappa - 1$.

Proposition 4 (One batch of identification). Consider a block of J batches of one concept, with log-losses clipped at c, and let Assumption 3 hold for every labelled batch of the block. Then the regret of the fast level per row, summed over the batches of the block, satisfies

$$\mathrm{Regret}_{\mathrm{ME}} \;\leq\; c + (J-1)(K-1)\, e^{-\eta g}.$$

Remark 4. The first term is one batch of identification, lost because labels arrive one batch late. On the row scale of the cumulative bound $\sqrt{JB \ln K}$ the regret is at most $Bc + B(J-1)(K-1)e^{-\eta g}$: linear in the length of the block, but with a coefficient that is exponentially small in the margin.

3.4 When Recall Beats Relearning

Let concepts recur in blocks of L rows, that is, $J = L/B$ batches, and let $a_k(n)$ be the accuracy learning curve of concept k, non-decreasing in n. On the c-th visit of concept k ($c \geq 2$) its stored expert holds $n_c = \min(M, (c-1)L)$ rows. The relearning learner resets its context at the block boundary; in the terms of Zhao et al. [5] it is the online expert, and the stored context is an offline expert that needed no training.

Theorem 1 (Recall versus relearn). Let Assumption 3 hold on every labelled batch of the block for the expert with the larger learning-curve value, that is, the stored expert if $a_k(n_c) \geq a_k(\min(jB, M))$ and the window otherwise, with $e^\kappa - 1$ below the top-two gap of that expert. On the c-th visit of concept k, the expected accuracy of the mixture minus that of relearning, averaged over the block, is

$$\Delta_{k,c}(L) = \frac{1}{J}\sum_{j=1}^{J-1}\Bigl[a_k(n_c) - a_k\bigl(\min(jB, M)\bigr)\Bigr]_+ \;+\; \zeta_{k,c}, \tag{7}$$

where $\zeta_{k,c} = (A_1^{\mathrm{mix}} - A_1^{\mathrm{rel}})/J$ is the difference on the first batch of the block, which both learners predict before any label of the block has arrived, so $|\zeta_{k,c}| \leq 1/J$. We call the sum the area term.

Corollary 2 (When memory pays). On the first visit $n_1 = 0$ and the area term of (7) is zero; on later visits it is zero whenever the learning curve is flat, $a_k(B) = a_k(M)$, and in general

$$\Delta_{k,c}(L) \;\leq\; \frac{M}{L}\,\bigl(a_k(M) - a_k(B)\bigr) + \frac{1}{J}.$$

Remark 5. Equation (7) is the area by which the learning curve lies below the accuracy of the stored expert. In the decomposition (5) it is the regret of the online expert minus that of the recalled expert: memory helps when this area exceeds the cost of the meta expert, and Proposition 4 makes that cost one batch, which the relearning learner loses as well. The value of memory is therefore a property of the backbone and the concept that can be measured before deployment, and Corollary 2 explains Observation 2: for concepts that a TFM learns from one batch, the area is zero.

3.5 The Slow Level: A Safeguard at Any Time

Assumption 3 fails on streams whose experts are close to each other. The batch-wise restart then follows noise: the expert that was best on the last batch need not be best on the next, which is the situation of Observation 3. MICE therefore places a second meta expert above the first. It has two experts, the default (the FIFO window of M rows, $k = 0$) and the fast mixture (6) ($k = 1$), with predictions $p_{t,0}$ and $p_{t,1}$ on batch t. Let $\ell_{t,k} \in [0, 1]$ be the mean half Brier score $\frac{1}{2}\|p - e_y\|_2^2$ of expert k on batch t. The slow level keeps discounted losses and predicts with

$$L_{t,k} = \gamma L_{t-1,k} + \ell_{t,k}, \qquad L_{0,k} = 0, \tag{8}$$

$$v_{t,k} \propto \exp\bigl(-\eta_2\, \gamma\, L_{t-1,k}\bigr), \tag{9}$$

$$p_t = v_{t,0}\, p_{t,0} + v_{t,1}\, p_{t,1}, \tag{10}$$

with a discount $\gamma \in (0, 1]$. For $\gamma = 1$ this is the cumulative meta expert of MOOE with an exp-concave loss; $\gamma < 1$ forgets losses older than about $1/(1 - \gamma)$ batches.

Theorem 2 (Any-time safeguard). Let K experts be combined by (8)–(10) with a step $\eta_2 \le \frac{1}{2}$. For every sequence of labels and expert predictions, every time T and every expert k,

$$\sum_{t=1}^{T} \gamma^{\,T-t} \bigl(\ell_t(p_t) - \ell_{t,k}\bigr) \;\le\; \frac{\ln K}{\eta_2}. \tag{11}$$

Corollary 3 (Cost of following the fast level). With $K = 2$, $\eta_2 = \frac{1}{2}$ and $\gamma = 0.9$, the discounted regret of MICE against the plain window, and against the fast mixture, is at most $2 \ln 2 \approx 1.39$ at every time, against a discounted horizon of $1/(1 - \gamma) = 10$ batches.

Remark 6. Theorem 2 makes no assumption on the stream. The discount makes it an any-time statement: the comparison covers an effective horizon of ten batches, not the length of the stream, and older losses fade geometrically. How fast the slow level follows the better of the two experts after a change depends on their earlier losses and on the loss gap after the change. The two levels divide the work. The fast level identifies a recurring concept within one batch when there is something to identify (Proposition 4); the slow level decides whether identification is worth following. The guarantee is proved for the half-Brier loss with $\eta_2 \le \frac{1}{2}$, where it is exp-concave; being bounded, it also limits how far one batch moves the weights. The log-loss is exp-concave but unbounded, and in our runs one confidently wrong batch after a switch dominated the slow level for the whole horizon (Section 5.4).

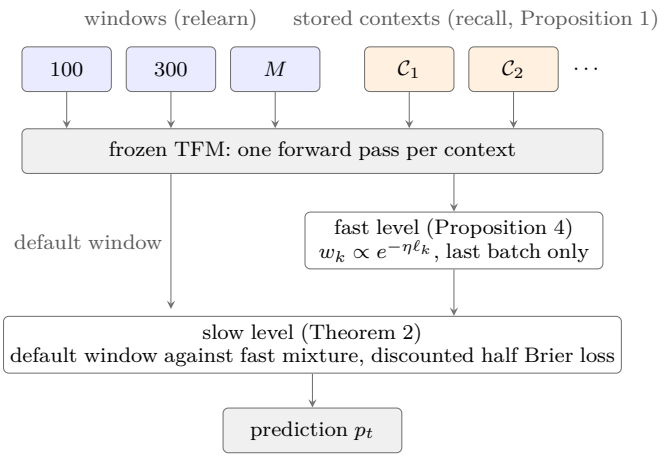


Fig. 1. MICE for one batch. Windows and stored contexts are in-context experts of the same frozen TFM. The fast level mixes all experts from their losses on the last labelled batch; the slow level mixes the result with the default window. Each component is governed by one result of Section 3.

4 Method: A Mixture of In-Context Experts

MICE (Figure 1 and Algorithm 1) runs a frozen TFM; no expert is trained. Table 1 collects the notation. Each component implements one result of Section 3.

Experts. Windows are FIFO contexts of the last 100, 300 and M labelled rows; they relearn fast and cover concepts that are new or cheap to learn. Stored concepts are contexts kept in a pool, built from segments of the stream that start at detected changes (Section 3.2). After every labelled batch the errors of the 300-row window are tested by (4) with $H_{\mathrm{cp}} = 3$, $R_{\mathrm{cp}} = 5$ and $\delta_{\mathrm{cp}} = 10^{-3}$. When a change is declared at batch b, the open segment ends at row bB and is closed if it holds at least 300 rows (shorter ones stay in the windows only), and the next segment starts there. Without a change a segment closes after 500 rows, once three further batches are labelled, so that a change in its last rows can still end it first; a segment therefore holds 300 to 700 rows. The test uses only the window's errors, so the segments do not depend on the pool. We call fixed segmentation the variant that closes a segment every 500 labelled rows from the first row of the stream; it is the original form of MICE, and the experiments name it where they use it. A closed segment S joins the stored expert $\mathcal{C}_k$ that minimises the estimated recall regret $\widehat{\mathrm{D}}_T(S \,|\, \mathcal{C}_k) = \hat{\epsilon}_S(\mathcal{C}_k) - \hat{\epsilon}_S(S)$ if this is at most $\delta = 0.02$, where $\hat{\epsilon}$ is the 0-1 risk (one minus accuracy, as in Proposition 1(iii)) and $\hat{\epsilon}_S(S)$ is cross-fitted in two folds because an in-context learner memorises its context; otherwise it founds a new stored expert (Proposition 1). A stored expert keeps its last M rows, and the pool holds at most 12 experts (least recently used out).

Fast level. The windows and the stored experts are mixed with the weights (6), $\eta = 10$ (Lemma 1 and Proposition 4).

Slow level. The fast mixture and the M-row window are mixed by (8)–(10) with $\gamma = 0.9$ and $\eta_2 = \frac{1}{2}$ (Theorem 2 and Corollary 3).

Timing. Every loss scores a prediction that was issued before its labels arrived (Algorithm 1): each expert's prediction for a batch is cached when the batch is predicted, and when

Algorithm 1 MICE for one batch $t \geq 1$

1: Initialisation (before $t = 1$): batch 0 is observed with its labels and predicted by no one; the windows hold its rows, the pool is empty, and its rows count towards the first segment (rows are counted from the first row of the stream)
2: Input: unlabelled batch X_t; labels y_{t-1} of batch $t-1$; windows $\mathcal{W}$; pool $\mathcal{P}$; cached predictions $\hat{p}_{t-1,k}$ of every expert k, $\hat{p}^{\text{fast}}_{t-1}$ and $\hat{p}^{\text{def}}_{t-1}$, all issued for batch $t-1$ before y_{t-1} arrived
3: if $t = 1$: no prediction has been issued; set $\ell_k \leftarrow 0$ for every expert (uniform fast weights) and $L_{0,\text{def}} = L_{0,\text{fast}} \leftarrow 0$ (uniform slow weights), and go to the context update
4: Score issued predictions ($t \geq 2$): $\ell_k \leftarrow$ mean log-loss of $\hat{p}_{t-1,k}$ on y_{t-1}; $\ell^{\text{def}}, \ell^{\text{fast}} \leftarrow$ mean half Brier score of $\hat{p}^{\text{def}}_{t-1}$, $\hat{p}^{\text{fast}}_{t-1}$; $L_{t-1,k} \leftarrow \gamma L_{t-2,k} + \ell^k$ for $k \in \{\text{def}, \text{fast}\}$
5: Update contexts: add batch $t-1$ to the windows; record the errors of the 300-row window on it; if (4) declares a change, or the open segment has 500 rows and three more batches are labelled: close a segment S (if it has at least 300 rows) and merge it into $\arg\min_{k \in \mathcal{P}} \widehat{\mathrm{D}}_T(S \,|\, \mathcal{C}_k)$ if the minimum is at most δ (the expert keeps its identity), else add S to $\mathcal{P}$ as a new expert k' with $\ell_{k'} \leftarrow$ median of the ℓ_k
6: Predict: $\hat{p}_{t,k} \leftarrow f_\Theta(\cdot \mid X_t; \mathcal{C}_k)$ for $k \in \mathcal{W} \cup \mathcal{P}$; $\hat{p}^{\text{fast}}_t \leftarrow \sum_k w_k \hat{p}_{t,k}$ with $w_k \propto e^{-\eta \ell_k}$; $\hat{p}^{\text{def}}_t \leftarrow \hat{p}_{t,M}$ (the M-row window)
7: Output and cache: $\hat{p}_t = v_0\, \hat{p}^{\text{def}}_t + v_1\, \hat{p}^{\text{fast}}_t$ with $v_k \propto e^{-\eta_2 \gamma L_{t-1,k}}$; cache the expert predictions $\hat{p}_{t,k}$ (with $\hat{p}^{\text{def}}_t = \hat{p}_{t,M}$; $\hat{p}^{\text{fast}}_t$ is a function of them and of earlier labels)

TABLE 1
Notation.

Symbol	Meaning
$(\mathbf{x}, y)$	a row: feature vector $\mathbf{x}$ and label $y \in \mathcal{Y}$
B, M	rows per batch ($B = 100$); context budget in rows ($M = 1000$)
$\mathcal{C}$, $\mathcal{C}_k$	a context (set of labelled rows); the context of stored expert k
$f_\Theta(\cdot \mid \mathbf{x}; \mathcal{C})$	frozen TFM with parameters Θ: maps a row $\mathbf{x}$ and a context $\mathcal{C}$ to a distribution over $\mathcal{Y}$
P, Q; P_X; $p_{\mathbf{x}}$	concepts (distributions over $(\mathbf{x}, y)$); marginal of $\mathbf{x}$; conditional $P(\cdot \mid \mathbf{x})$
$\epsilon_P(\mathcal{C})$, $R_P(n)$	risk on P of the TFM given $\mathcal{C}$; learning curve, the expected risk for n rows of P
D_T, D_E, D_R	recall regret and excess-risk discrepancy (Definition 2); R-divergence [15]
S, δ	a closed segment (300 to 700 rows); merge threshold ($\delta = 0.02$)
z_t, τ	errors of the 300-row window on batch t; margin of the change test (4)
H_{cp}, R_{cp}, δ_{cp}	batches after and before a candidate change (3, 5); level of the test (10^{-3})
π, β, ν	majority share of a context; error gap after a change; rows between a declared change and the boundary
K	number of experts of a meta expert; in Definition 1, number of past intervals
ℓ_k, η, w_k	log-loss of expert k on the last labelled batch; fast step ($\eta = 10$); fast weight
g, κ	margin of the leading expert; $\kappa = \ln\big(1 + (K-1)e^{-\eta g}\big)$
L, J	block length in rows; batches per block, $J = L/B$
$a_k(n)$, n_c	accuracy learning curve of concept k; rows held by its stored expert on visit c
$\Delta_{k,c}(L)$, $\zeta_{k,c}$	accuracy of the mixture minus that of relearning on visit c; first-batch term
$\ell_{t,k}$, $L_{t,k}$	half Brier score of slow-level expert k on batch t; its discounted cumulative loss
η_2, γ, $v_{t,k}$	slow step ($\eta_2 = \frac{1}{2}$); discount ($\gamma = 0.9$); slow weight

the labels come one batch later the cached predictions are scored; contexts are updated only afterwards, so no expert is scored on rows it already holds. The fast level uses the log-loss of the last labelled batch only. A stored expert created at a segment close has issued no prediction for that batch and starts with the median of the other experts' losses; a merged expert keeps its identity and its losses. Batch 0 only fills the windows and counts towards the first segment; at $t = 1$ nothing has been issued yet, so the fast weights are uniform and $L_{0,k} = 0$ makes the slow weights uniform, and scoring of cached predictions begins at $t = 2$, where $L_{1,k} = \gamma L_{0,k} + \ell_{1,k}$. The slow level is scored on the default and fast predictions actually issued, with the mean half Brier score in $[0, 1]$; these are the two adaptive prediction sequences of Theorem 2, whose bound needs $\eta_2 \leq \frac{1}{2}$ for the half Brier score. It therefore covers the method as run, but not the variants we report with the log-loss or with the registered step $\eta_2 = 10$.

The cost is one TFM call per expert and batch, at most 15, plus, when a segment closes, two cross-fitted calls and one call per stored expert (at most 14), so at most 29 calls on a closing batch; at most one segment closes per batch, and the change test reuses the window's predictions and makes no call.

5 Experiments

The experiments test the theory in its two regimes. Section 5.1 asks whether learning curves predict the gains of recall; Section 5.2 how pure segments at detected changes are and whether they keep the gain of recall wherever the boundaries fall; Section 5.3 whether the slow level keeps MICE close to the plain window where there is little to recall; and Section 5.4 what the organisation of the pool, probability quality and resources contribute.

Protocol. Prequential evaluation in batches of $B = 100$ rows with labels one batch late; context budget $M = 1000$ for every policy; backbone TabPFN v2 with four ensemble members [6], and TabICL [7] as a second backbone.

Segmentation. MICE with detected segments and with fixed segmentation were both run on the twelve grid streams of seeds 20 and 21, on the streams of Section 5.2 and on the 29 real streams; on the aligned grid streams with 30 or 100 centroids and on the real streams they differ by $+0.04$ and $+0.01$ points on average (Table 9). The other results (generator streams, oracle test, second backbone seed, TabICL, ablations, pool controls and timing) were obtained with fixed segmentation, except the resource and snapshot comparison of Table 5, which uses detected segments.

In-context baselines: FIFO; LTM [10]; DDM-triggered reset [12]; a window ensemble of the three windows of MICE

without the pool. Other stream learners: Adaptive Random Forest (ARF) [16], Streaming Random Patches (SRP) [17], the Hoeffding Adaptive Tree [18], Leveraging Bagging [19], Adaptive Windowing (ADWIN) Bagging [20], Learn++.NSE [21], and MOOE with trained experts [5], tuned on the development streams over 72 configurations and an extension of its grid (Appendix E).

Streams. A controlled grid of recurring concepts: mixtures of 5, 30 or 100 Gaussian centroids in ten dimensions with four nominal classes, where each of three concepts assigns its own, independently drawn class to every centroid, so that P_X is fixed and concepts conflict on the centroids where their classes differ; blocks of 500 or 2000 rows; three cycles (Appendix F gives the full specification). Thirty streams from six standard generators with recurring concepts, as implemented in River [22]: SEA [23], STAGGER [13], Agrawal [14], Sine [12], random radial basis function (RBF) [20] and Hyperplane [24]. Twenty-nine real streams from 13 sources, the long ones in segments of 100,000 rows: Electricity and seven INSECTS streams [25] from River [22] (one INSECTS stream in three segments); Forest CoverType[1] in five segments; and, from a public collection of drift data sets,[2] PokerHand and Airlines in five segments each, Rialto [26], NOAA Weather [21], Spam [27] and Phishing.

Development and confirmation. The real streams were opened in five stages (Table 4): the first three developed the slow level, and the last two test it: the fourth set was registered for the method of Section 4 with fixed segmentation, which on the third set was added after the baselines of three segments were visible (Appendix E).

5.1 Learning Curves Predict Recall Gains

Theorem 1 predicts the gain of recall from the learning curves of the grid concepts, measured before the grid streams were run (Appendix F). On seeds 10 and 11 the fast level alone exceeds reset on all eight streams with 30 or 100 centroids, more for shorter blocks and for harder concepts, as predicted, and the rank correlation between predicted and observed gain over the six cells is 0.94; the magnitudes agree as well (0.55 versus 0.53 points, 3.0 versus 2.6, 10.4 versus 8.6). On the fresh seeds 20 and 21 with the full two-level method the correlation is 0.83 (Table 2 and Figure 2). On these aligned streams the segments of both segmentations hold one concept, and detected segments change the accuracy by less than 0.2 points on each of the twelve streams (Section 5.2). The predictions in Table 2 and Figure 2 assume M stored rows on every recurrence; with the visit-specific size $n_c = \min(M, (c-1)L)$ of Theorem 1 they change only for $L = 500$ (100 centroids: 9.2 instead of 10.4 points, closer to the observed 8.6) and the rank correlations are unchanged. These observed gains are against DDM, whose detection delay, and the slow level of MICE, lie outside Theorem 1. The quantity of the theorem itself is measured in Table 3: with the boundaries and concept identities of the generator, relearning resets its context at the block boundary, the stored expert holds the last n_c rows of the concept's earlier visits, and the mixture of the two weights them by $\exp(-\eta\,\ell)$ with $\eta = 10$ and ℓ the log-loss on the last

1. https://archive.ics.uci.edu/dataset/31/covertype
2. https://github.com/ogozuacik/concept-drift-datasets-scikit-multiflow

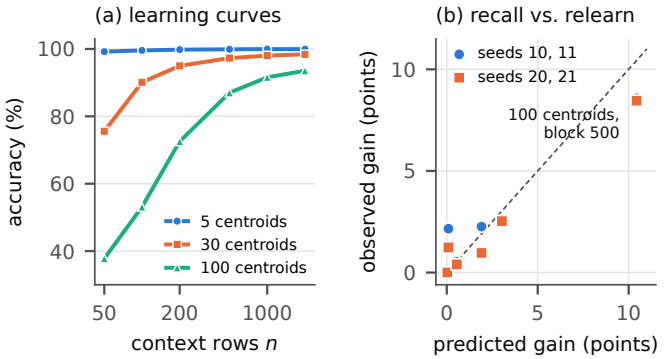


Fig. 2. (a) Learning curves of the grid concepts with TabPFN v2, measured before the grid streams were run. (b) Gain of memory over reset per grid cell: predicted from these curves (registered values, see Section 5.1) against observed, for the fast level on seeds 10 and 11 and for MICE on seeds 20 and 21; the dashed line is equality.

labelled batch. On seeds 40 and 41, which no earlier test had used, the measured area term lies within 0.7 points of the visit-specific prediction in every cell (mean absolute difference 0.37 points; Spearman correlation 1.0), from 0.3 against 0.02 points for 5 centroids and $L = 500$ to 8.7 against 8.9 for 100 centroids and $L = 500$. Identification follows the margin of Assumption 3: on the 74 batches where the two experts differ by more than 10 points the mixture puts more than half of its weight on the more accurate one every time, on the 59 batches with a difference of 5 to 10 points on 90% of them, and on the 221 batches within 2 points of each other on 49%; weighted by the accuracy difference, the agreement is 0.87. The net cost against the better expert on every batch is 0.03 points, with batches on which the mixture is more accurate than both offsetting the others. On the first batch of a block the weights come from the last batch of the preceding block, which belongs to another concept; the stored expert received a weight of at most 3×10^{-10} there, the mixture was as accurate as relearning on all 72 first batches of recurrences, and the measured first-batch term is zero. On seeds 20 and 21 the gain of the deployed MICE over DDM is below the oracle area in five of the six cells, by 0.06 to 1.14 points; this departure combines DDM's detection against the oracle reset, MICE's memory and identification without oracle boundaries, and the slow level. On the sixth cell (5 centroids, $L = 500$) the gain over DDM is 1.24 points against an oracle area of 0.03. On seeds 10 and 11 the curves were measured on the evaluated concepts; on seeds 20 and 21 they predict the gains of new concepts from the same generator. The six standard generators are the other end of the scale: their learning curves are flat after about 200 rows, the predicted gain is at most 0.4 points, and the observed gain of MICE over reset is 0.1 points on average. This is the low-area regime of Theorem 1: memory has little to add where relearning is cheap.

On the grid MICE exceeds reset on all eight streams with 30 or 100 centroids and the window ensemble on all eight (Table 2): the gain comes from memory, not from faster adaptation. Averaged over the twelve streams it reaches 86.5 against 84.3 for reset, 81.8 for the window ensemble and 62.4 for FIFO; on the hardest cell the gain over reset is 8.5 points. The slow level costs 0.4 points here against the fast level alone,

TABLE 2
Controlled grid, fresh seeds 20 and 21 (accuracy in %, mean of two streams per cell). Win.: window ensemble; Fast: the fast level without the slow level. Pred.: gain over reset predicted from learning curves measured beforehand (registered values, which assume M stored rows on every recurrence; see Section 5.1); Gain: observed MICE minus reset, in points.

Cent.	Block	FIFO	Win.	Reset	Fast	MICE	Pred.	Gain
5	500	52.3	82.1	87.3	88.7	88.6	0.1	+1.2
5	2000	85.2	96.3	96.8	96.9	96.8	0.0	+0.0
30	500	40.2	78.8	81.2	82.7	82.2	1.9	+1.0
30	2000	79.3	92.5	93.6	94.3	94.0	0.5	+0.4
100	500	43.0	60.0	63.5	72.4	71.9	10.4	+8.5
100	2000	74.5	81.2	83.2	86.4	85.7	3.0	+2.5
Mean		62.4	81.8	84.3	86.9	86.5		+2.3

TABLE 3
Direct test of Theorem 1 with oracle boundaries (points). Pred.: area term from the learning curves with $n_c = \min(M, (c-1)L)$; Area: measured mixture minus relearning over batches $2, \ldots, J$ of each block, divided by J and averaged over all blocks; Best: the better of stored expert and relearning on each batch in place of the mixture; MICE−DDM: deployed gain on the same seeds. Two streams per cell.

			Seeds 40, 41		Seeds 20, 21	
Cent.	Block	Pred.	Area	Best	Area	MICE−DDM
5	500	0.02	0.32	0.46	0.03	1.24
5	2000	0.01	0.21	0.31	0.06	0.01
30	500	1.70	2.08	2.06	1.62	0.97
30	2000	0.52	0.96	1.02	0.81	0.40
100	500	8.91	8.68	8.81	9.11	8.45
100	2000	2.83	3.52	3.27	3.67	2.53
Mean abs. diff. to Pred.			0.37	0.35	0.25	

the price of the safeguard. Stream learners that train a model are far behind when a concept must be learned within a block: on seeds 10 and 11, ARF reaches 68.0, SRP 69.7 and the tuned MOOE 78.1, against 83.9 for reset and 86.2 for MICE. On the thirty generator streams MICE matches reset (95.0 against 94.8), and the forgetting of Observation 1 is repaired: 0.82–0.89 in the five batches after a recurrence against 0.17–0.51 for FIFO.

5.2 Segment Purity and Boundary Phase

Theorem 1 assumes stored contexts whose rows come from one concept, and Proposition 2 shows that in the population limit a majority identifies one of two deterministic concepts with a shared input distribution. Fixed segmentation gives pure contexts only where concept boundaries coincide with segment ends, as on the grid of Section 5.1, where every concept lasts 500 or 2 000 rows from row zero. Three tests of fixed segmentation, with the same baselines and constants, remove this coincidence; they motivate segmentation at detected changes, which we then test on fresh streams.

Fixed segmentation. (a) Misaligned schedules. Twelve streams of the same generator (data seeds 30 and 31, 30 or 100 centroids, three concepts in nine blocks) have blocks of 700 rows, of 1 300 rows, or drawn uniformly from $\{500, 600, \ldots, 2\,500\}$ rows, so that concept boundaries fall inside the 500-row segments (7 of the 8 boundaries for blocks of 700 and 1 300 rows). Fixed segmentation is above reset on 8 of the 12, by +0.47 points on average (Table 6), short of the registered criterion (Appendix E). With 100 centroids it stays above reset on all six streams (+0.8 to +1.9 points; +7.1 to +10.9 in the first five batches of each recurring block); with 30 centroids it is between −0.7 and +0.3 (−2.3 to +2.5 in those batches). A pool that keeps every closed segment as its own expert, without the discrepancy test and without merging (an archive), is more accurate on all twelve streams, by 0.2 to 1.8 points.

(b) Fixed block length, shifted boundaries. The eight aligned grid streams of Table 2 with 30 or 100 centroids, with 200 rows of the first concept prepended (drawn from the same centroids with a separate random generator): every recurring block keeps its length, and every boundary lies 200 rows into a 500-row segment while staying on a batch boundary. Batch $t + 2$ of a shifted stream holds exactly the rows of batch t of its aligned stream, so the two are compared on matched batches. The shift leaves FIFO, reset and the window ensemble almost unchanged (within one point) but removes the recall advantage (Table 7): the gain over reset changes by −6.8 points on average (−1.0 to −14.4; −12.0 in the first five batches of the recurring blocks), and fixed segmentation is below reset on 7 of the 8 shifted streams (−3.7 points on average, down to −10.8 with blocks of 500 rows), short of the registered criterion. With 500-row blocks it is also below the window ensemble (−0.9 to −8.5 points); with 2 000-row blocks it stays above it (+0.2 to +1.8) and loses 1.0 to 3.0 points of its advantage over reset, and with 100 centroids it keeps +5.7 and +7.3 points over reset in the first five batches of the recurring blocks. The archive loses less and is above fixed segmentation on all eight shifted streams (by 3.0 points on average), whereas on the aligned streams it is 0.3 points below.

(c) Shifted boundaries, segments realigned. Rerunning on the shifted streams with the first segment starting at row 200, using the known phase of the boundaries, makes every stored segment hold one concept again while the prepended rows still enter the windows. This restores the recall advantage (Table 7): above reset on all eight streams, by 3.01 points on average against 3.09 on the aligned streams, and 0.29 points above the archive against 0.30. Restoring the phase changes the gain over reset by +6.7 points on average (+12.0 in the first five batches of the recurring blocks), while the prepended history changes it by −0.08 points.

The three tests locate the loss in the phase of the segments relative to the concept boundaries: with 500-row blocks every fixed segment mixes two concepts, with 2 000-row blocks one in four, and with the same rows and the segments realigned the advantage returns. Segmentation at detected changes seeks this alignment without knowing the phase.

Detected segmentation on fresh streams. The test streams are 72 new grid streams (registered before any run, Appendix E): four data seeds (40 to 43), 30 or 100 centroids, and three length regimes, nine blocks of 500 rows, of 2 000 rows, or of lengths drawn uniformly from 500 to 2 500 rows; each combination is run with three independently drawn offsets of 1 to 499 rows (1 to 1 999 for 2 000-row blocks) of the first concept prepended, so that boundaries fall at arbitrary rows, mostly inside batches. Seeds 40 and 41 had been used for the oracle test of Table 3, which involves no detector and set no constant, so their concepts, and for fixed block lengths their rows after the offset, had been seen; the

TABLE 4
Real streams: mean accuracy (%) per evaluation stage, and the worst difference to FIFO over the streams of the stage (points). Per-stream results are in Table 17.

		Mean accuracy						Worst difference to FIFO			
Stage	Streams	FIFO	LTM	Reset	Win. ens.	Fast only	MICE	Reset	Win. ens.	Fast only	MICE
development	5	76.6	76.5	76.8	77.1	77.6	77.6	−1.1	−0.7	−0.4	+0.3
first held-out set	4	83.1	82.8	83.3	83.3	83.8	83.8	−0.4	−0.3	−0.2	+0.3
second held-out set	6	84.9	84.3	83.5	84.8	85.1	85.7	−8.1	−1.4	−2.3	−0.2
third held-out set	8	84.9	84.1	82.3	84.4	84.6	85.2	−8.4	−1.2	−1.7	−0.1
fourth held-out set (confirmatory)	6	85.1	83.9	80.9	84.2	84.5	85.3	−9.6	−1.4	−2.0	−0.0
all streams	29	83.3	82.6	81.5	83.0	83.3	83.8	−9.6	−1.4	−2.3	−0.2

detector constants were set on development seeds 0 and 1 (Appendix E). Uncertainty is estimated by a hierarchical bootstrap that resamples seeds within each centroid count and offsets within each seed. Table 8 gives the results. MICE is above reset on 70 of the 72 streams, by +2.85 points on average, and in every length regime: +5.17 points with 500-row blocks (24 of 24; 95% interval $[+4.59, +5.72]$), +1.48 with 2 000-row blocks (22 of 24; $[+1.22, +1.72]$) and +1.91 with variable blocks (24 of 24; $[+1.74, +2.07]$). Fixed segmentation gains +0.66 ($[-2.34, +3.31]$), +0.74 and +0.65 on the same streams. In the first five batches of the recurring blocks the gain over reset is +10.1 points against +5.3. On seeds 42 and 43 alone (36 streams, a split made after the evaluation) MICE is above reset on all 36, by +2.84 points, and +5.35, +1.41 and +1.76 in the three regimes. On 72 confirmation streams from seeds 50 to 53, with the same construction and criterion, MICE is above reset on all 72, by +2.69 points: +5.07 ($[+4.31, +5.83]$), +1.35 ($[+1.20, +1.51]$) and +1.64 ($[+1.48, +1.80]$) in the three regimes, against −1.31, +0.39 and +0.43 for fixed segmentation. The ordering of the cells is that of the learning curves (Section 5.1): with 100 centroids the gain is +2.4 to +8.2 points, with 30 centroids +0.5 to +2.2.

On the streams of tests (a) to (c), detected segmentation recovers what fixed segmentation lost and leaves the rest unchanged (Table 9): +6.55 points over fixed segmentation on the shifted grid (8 of 8), where it is +3.07 points above reset against +3.13 on the aligned streams, and +1.74 on the misaligned schedules (12 of 12); on the aligned grid the two differ by +0.04 points and on the 29 real streams by +0.01 ($[-0.02, +0.05]$; −0.37 to +0.41 on single streams).

Detection, location and purity. We measure the three conditions of Section 3.2 separately. The detector declared 570 of the 576 changes of the 72 streams, and 575 of the 576 on the confirmation streams, at the batch that holds the boundary or at one of the three batches after it, and made one cut elsewhere (none on the confirmation streams), consistent with Proposition 3. Each of the 570 and 575 declarations matched to a boundary lay in the batch that holds it (59% and 64%) or in the next one, so within $\nu < B$ rows of the boundary (median 24 and 28 rows, at most 78), the location condition of Corollary 1. The premises of Corollary 1 thus held on 67 of the 72 test streams and on 71 of the 72 confirmation streams; the others have a missed boundary, and one of them also the cut away from any boundary. At every segment close we measured the share of the majority concept among the rows of the stored context that received the segment. All closes were majority-pure. The grid concepts are not deterministic, as their Gaussian components overlap, so this share describes the mixing and does not verify the identification of Proposition 2; 88% held at least 95% rows of one concept (87% on the confirmation streams), short of the registered 90% (Appendix E), and the mean share was 97.6%. The impure closes concentrate on 500-row blocks (69% at the 95% level, against 92% and 88% for the other regimes), where a boundary inside a batch leaves up to 99 rows of the neighbouring concept in a segment of at least 300. The stored contexts are thus majority-pure but not pure: Theorem 1 is not verified for them, and the gains above measure how much of this mixing recall tolerates. On the real streams the detector declares between 0 and 180 changes per stream (median 9), where Assumption 2 need not hold; there the slow level keeps MICE within 0.23 points of FIFO on every stream (above it on 24 of 29, +0.58 points on average).

Archive and resources. An archive with the same detected segments is 0.11 points below MICE on the fresh streams (MICE above it on 50 of 72) and 0.08 below on the real streams, and holds 7.64 stored experts with 10.64 TFM calls per batch on the fresh streams, against 2.42 experts and 6.26 calls for MICE (6.56 calls with fixed segmentation). On the real streams MICE makes 10.0 calls per batch on average (4.6 to 17.3). With detected segments the advantage of the archive in tests (a) and (b) disappears, and merging by discrepancy keeps the pool at a third of its size.

Resources and the snapshot pool with detected segments. In the isolated benchmark of Table 12, run with detected segments (Table 5), MICE takes 1.07 s per batch on the three real prefixes, with 2.0 stored experts, 5.6 TFM calls and 1 900 stored rows, and the archive with the same segments 2.58 s, with 10.7 experts, 13.4 calls and 5 200 rows; on four grid streams (100 centroids; blocks of 500, 2 000 and 500 to 2 500 rows, and shifted 500-row blocks) the times are 1.03 and 1.58 s, and the archive is slower on each stream (1.09 to 1.93 times). MICE has the higher peak of reserved GPU memory (1.15 against 0.76 GB on the prefixes). The snapshot pool needs no detector: every 500 rows it stores the latest 1 000 rows. Of the pool sizes 2 and 4, none stays within the time per batch of MICE on all three prefixes; two experts take 0.95 s on average, but 1.08 against 0.96 s on CoverType, and this smaller size is used. With two experts the snapshot pool is below MICE on all 72 confirmation streams, by 5.69 points on average (95% interval $[+4.98, +6.49]$; 9.44, 3.29 and 4.33 in the regimes of 500-row, 2 000-row and variable blocks), by 6.55 points on the shifted grid and by 6.61 on the misaligned schedules; with four experts by 5.06, 6.46 and 5.04 points. With twelve experts it has the pool capacity of MICE and takes more time

per batch: 2.78 s on the prefixes (2.6 times) and 1.69 s on the grid streams (1.28 to 2.04 times). MICE is then 2.14 points more accurate on the confirmation streams ($[+1.40, +2.96]$, above it on 50 of 72), 3.42 on the shifted grid and 1.32 on the misaligned schedules, and level with it on 2 000-row blocks (-0.13, $[-0.35, +0.12]$, above it on 9 of 24), where at least half of the snapshots lie inside one block and hold a single concept, against none with 500-row blocks. Against MICE with detected segments, a snapshot of the latest rows thus matches recall only on long blocks, and there at about twice the time per batch.

5.3 The Slow Safeguard on Real Streams

Real streams rarely announce a recurrence, and the experts are close to each other. Here the requirement is safety first. Table 4 shows that every policy that reacts to the last errors pays for it somewhere: reset loses up to 9.6 points to FIFO (PokerHand) and 1.8 on average, the window ensemble up to 1.4, and the fast level alone up to 2.3. MICE loses at most 0.25 points on any of the 29 streams, is above FIFO on 24 of them (+0.57 on average), and is more accurate than all four in-context baselines on 23; with detected segments it loses at most 0.23 points and is above FIFO on 24 (+0.58; Section 5.2). The rest of this section uses fixed segmentation. Its largest gains are where the stream revisits its past, Rialto (+4.7 points over FIFO, with a daily cycle) and the INSECTS streams (+0.2 to +1.9).

Confirmatory sets. On the fourth held-out set MICE is not below FIFO or reset on any of the six segments (worst difference to FIFO -0.01 points), it is more accurate than all four in-context baselines on five segments and has the highest mean (85.3 against 85.1 for FIFO, 84.2 for the window ensemble and 80.9 for reset), and it exceeds its own variant without the pool on all six. The fast level alone is 2.0 and 1.9 points below FIFO on the two PokerHand segments, which the slow level turns into +0.4 and +0.5; reset is 9.6 and 8.6 points below. On the third held-out set, MICE is within 0.08 points of FIFO on every segment, has the highest mean (85.2 against 84.9 for FIFO) and is the most accurate policy on five of eight segments; the paired difference to FIFO over batches is +0.30 points (block-bootstrap 95% interval $[+0.20, +0.40]$), and the registered step of 10 gives the same worst case and five of eight segments.

Uncertainty. To account for sampling variability and for dependence between consecutive batches, we computed for every real stream the one-sided 95% lower bound of the mean per-batch accuracy difference to FIFO with a moving-block bootstrap over batches (blocks of 20 batches, 10 000 resamples, percentile bound; the bootstrap treats the per-batch differences as locally stationary, so the 95% level is approximate under drift). The lower bound is above -0.3 points on 28 of the 29 streams, and blocks of 10 and 50 give the same count. On the six segments of the fourth held-out set the worst lower bound is -0.029 points (Covertype-e; -0.030 with blocks of 50). The exception is Spam, which has only 61 batches: its mean difference is -0.25 points with a lower bound of -0.85, so non-inferiority there is undecided rather than refuted. For the unweighted mean over the streams of each of the nine sources, with every stream resampled independently, the worst lower bound apart from Spam is -0.139 points (Weather), and Spam's is -0.85. These bounds concern backbone seed 0 on fixed streams; the second seed below varies the model, not the data, and segments of one source are not independent evidence.

A second seed. We repeated ten segments with another seed of the backbone. On the fourth held-out set the results hold again (most accurate on five of six segments, never below FIFO). On four segments of the third held-out set, non-inferiority, safety and the highest mean hold, and MICE is the most accurate policy on two of the four segments, where the hypothesis asked for three. Over the ten segments the difference to FIFO has the same sign under both seeds and changes by at most 0.4 points; its minimum is -0.08 points.

Slow level. It leaves the recall regime almost unchanged (grid: 86.5 against 86.9 for the fast level alone) and removes the losses on real streams (worst case -0.25 against -2.3; Table 4). With the log-loss in place of the half Brier score, one grid stream falls from 86.4 to 74.9: after a switch the fast mixture is confidently wrong for one batch, and an unbounded loss lets that batch outweigh the next ten.

Margin. On the grid the leading expert holds weight 1.000 from the second batch of a recurring block, with a gap of 0.8–8.0 nats to the runner-up, so $(K-1)e^{-\eta g} < 0.005$ in Proposition 4; on the real streams the median gap is 0.008–0.012 nats, the regime for which the slow level is built.

Regret. Over 33 development streams the largest discounted regret of the slow level is 0.15 against the window and 0.35 against the fast mixture, within the bound of 1.39; on adversarial sequences the bound holds at $\eta_2 = \frac{1}{2}$ and fails for steps of 2 and 10 (Appendix E).

Other learners and a second backbone. On all 29 streams, the best trained stream learner per stream (seven learners, MOOE in two configurations) is below MICE on 28 and below FIFO on 23; the exception is one Airlines segment, where Streaming Random Patches is 0.3 points above MICE. On CoverType and PokerHand the trained learners are 4 to 19 points behind. With TabICL as the backbone and no constant changed, MICE is the most accurate policy on all four streams of the first held-out set (84.0 on average against 83.4 for FIFO and 83.5 for the window ensemble), and on four segments of the third held-out set and all six of the fourth the results match those with TabPFN: on the fourth set MICE is the most accurate policy on six of six segments (85.2 on average against 84.7 for FIFO, 84.2 for the window ensemble and 81.3 for reset), and over the ten segments it is at most 0.02 points below FIFO.

5.4 Pool Organisation, Probability Quality and Resources

Windows and pool. Removing the pool, so that the two levels mix the three windows only, lowers accuracy on every one of the 24 grid streams (by 5.6 points on average) and on 28 of the 29 real streams (by 0.35 points on average, 2.1 on Rialto and 1.5 on a PokerHand segment): memory, not the choice of a window, carries the gain, on real streams as well. Removing the two short windows instead lowers the generator streams from 95.0 to 93.6 and the grid from 86.4 to 81.8 and leaves the real streams within 0.2 points on average; the pool forms exactly as many stored experts as a generator stream has concepts.

Pool controls. Removing the pool lowers accuracy, so past contexts carry the gain; two pools that never merge measure

TABLE 5
Resources and accuracy with detected segments. Mean stored experts, TFM calls and stored rows (thousands) per batch, mean and 95th percentile of seconds per batch, and peak reserved GPU memory (GB) from the isolated benchmark on the three real prefixes (mean over prefixes of the mean over three repetitions; one repetition for the snapshot pool with $k = 12$); mean seconds per batch on four grid streams (three repetitions); accuracy difference MICE minus snapshot pool (points) on the 72 confirmation streams (hierarchical bootstrap), the 8 shifted grid streams and the 12 misaligned schedules (moving-block bootstrap), with 95% intervals of the group mean. The archive uses the segments of MICE; its accuracy is in Table 9. The snapshot pool stores the latest 1 000 rows every 500 rows; $k = 2$ is the smaller of the sizes 2 and 4, neither of which stays within the time of MICE on all three prefixes; $k = 12$ is the pool capacity of MICE.

	resources							MICE minus snapshot pool, points		
Pool	experts	calls	rows, k	s, real	p95	s, grid	GB	confirmation	shifted	misaligned
MICE	2.0	5.6	1.9	1.07	2.09	1.03	1.15			
Archive, $k = 12$	10.7	13.4	5.2	2.58	3.11	1.58	0.76	–	–	–
Snapshot, $k = 2$	2.0	4.9	1.9	0.95	1.09	0.83	0.76	+5.69 [+4.98, +6.49]	+6.55 [+5.14, +8.34]	+6.61 [+5.87, +8.15]
Snapshot, $k = 4$	3.8	6.8	3.8	1.33	1.49	–	0.76	+5.06 [+4.53, +5.66]	+6.46 [+4.74, +8.33]	+5.04 [+4.46, +6.25]
Snapshot, $k = 12$	10.7	13.6	10.6	2.78	3.20	1.69	0.76	+2.14 [+1.40, +2.96]	+3.42 [+1.93, +4.62]	+1.32 [+0.88, +1.88]

TABLE 6
Misaligned schedules with fixed segmentation (test (a), registered): accuracy (%) and differences in points. Rec.: first five batches of each recurring block. Archive: every closed segment its own expert, no merging.

Blocks	cent.	seed	MICE	Reset	FIFO	Win. ens.	Archive	−Reset	−Reset (rec.)	−Archive
700	30	30	86.3	86.6	50.5	84.6	87.4	-0.27	+0.1	-1.11
700	30	31	86.8	86.7	52.8	83.5	87.9	+0.08	+1.2	-1.10
700	100	30	72.2	71.1	44.4	65.7	74.1	+1.13	+7.1	-1.82
700	100	31	72.9	71.0	48.0	66.8	74.7	+1.87	+9.0	-1.84
1 300	30	30	91.9	92.1	72.3	90.8	92.1	-0.21	+1.1	-0.20
1 300	30	31	91.5	91.8	72.0	90.6	91.7	-0.27	+0.3	-0.17
1 300	100	30	81.8	80.5	64.0	77.3	82.4	+1.28	+10.9	-0.66
1 300	100	31	81.7	80.9	66.4	77.2	82.7	+0.75	+9.4	-0.99
500–2 500	30	30	92.7	92.4	76.1	91.6	92.9	+0.29	+2.5	-0.22
500–2 500	30	31	93.8	94.5	81.3	93.6	94.5	-0.74	-2.3	-0.78
500–2 500	100	30	82.7	81.8	69.4	79.4	83.8	+0.87	+7.9	-1.14
500–2 500	100	31	85.2	84.4	75.6	82.5	85.9	+0.81	+8.4	-0.67
Mean			85.0	84.5	64.4	82.0	85.8	+0.47	+4.6	-0.89

TABLE 7
Fixed segmentation, block length fixed, boundaries shifted by 200 rows (tests (b) and (c), registered): MICE minus reset and minus the archive, in points, on matched batches of each aligned stream and its shifted copy. Δ: shifted minus aligned. Realig.: shifted stream with the segments starting at row 200. Rec.: first five batches of each recurring block.

			−Reset				−Reset (rec.)			−Archive		
Block	cent.	seed	aligned	shifted	Δ	realig.	aligned	shifted	realig.	aligned	shifted	realig.
500	30	20	+0.75	-7.66	-8.41	+0.61	+2.6	-9.5	+2.7	+0.02	-3.77	+0.02
500	30	21	+1.18	-10.75	-11.93	+0.91	+2.9	-14.2	+2.9	+0.02	-5.64	+0.02
500	100	20	+8.77	-2.57	-11.34	+8.84	+12.8	-3.9	+12.8	+0.39	-4.50	+0.39
500	100	21	+8.14	-6.27	-14.41	+7.98	+14.5	-6.3	+14.5	+1.07	-4.20	+1.07
2000	30	20	+0.51	-0.73	-1.24	+0.50	+4.0	-1.2	+4.0	+0.06	-0.77	+0.06
2000	30	21	+0.29	-0.69	-0.98	+0.28	+3.7	-1.0	+3.7	-0.02	-0.59	-0.02
2000	100	20	+2.89	+0.11	-2.78	+2.79	+16.2	+7.3	+16.2	+0.37	-2.09	+0.37
2000	100	21	+2.17	-0.80	-2.97	+2.16	+16.1	+5.7	+16.1	+0.45	-2.34	+0.45
Mean			+3.09	-3.67	-6.76	+3.01	+9.1	-2.9	+9.1	+0.30	-2.99	+0.29

how much the organisation of the pool adds. Under the same windows, pool size of 12, two-level rule and constants, an archive keeps every closed 500-row segment as its own expert, and a snapshot pool stores the latest $\min(1000, \text{rows so far})$ rows at every segment close. They are complete policies with their own budgets: all three make at most 15 TFM calls per batch; MICE and the snapshot pool hold at most 1000 rows per stored expert (13 400 context rows in all), the archive 500 (7 400); only MICE adds discrepancy calls when a segment closes. On the aligned grid (Table 10) MICE is above the archive on seven of the eight streams with 30 or 100 centroids, by +0.30 points (95% interval $[+0.12, +0.41]$, below the registered +0.5), and +3.3 points above the snapshot pool, mostly on 500-row blocks, where a 1000-row snapshot spans two concepts and the single-concept premise fails again. On the 29 real streams (Table 11) the three pools are close: MICE is +0.07 points above the archive (above it on 18 streams) and −0.08 points below the snapshot pool (above it on 11); the one-sided 95% block-bootstrap bounds are above −0.3 points on 23 and 17 streams, and single streams differ by up to 0.52 points. MICE keeps the smallest pool: 8.4 active experts per batch against 14.2, 9.7 TFM calls against 14.0 (fewer on 25 of the 29 streams, including about 1.3 discrepancy calls per batch), and 2 700 to 5 400 stored rows per batch on average (bounds from the 500- and 1000-row caps; per-expert counts were not recorded) against 5 600 for the archive and 11 100 for the snapshot pool; at the upper cap MICE can store more rows than the archive on single streams. Keeping past contexts carries the recall gain; organising them by discrepancy keeps the pool small in experts and calls.

Pool capacity at equal cost. At the same pool size of 12, MICE is within 0.3 points of both pools on the real streams and of the archive on the aligned grid, but not at the same cost. In an isolated benchmark on the three real prefixes of Table 13 (three repetitions), MICE takes 1.11 s per batch with 1.8 stored experts and 5.5 TFM calls on average, the archive 2.49 s (10.7 experts, 13.6 calls) and the snapshot pool 2.84 s (Table 12); on four grid streams (100 centroids, two aligned and two shifted) the times are 1.05, 1.58 and 1.63 s. Capping each control at the largest pool size among 2 and 4 whose time per batch does not exceed that of MICE, chosen from these timings alone, gives two experts for both (0.93 and 0.98 s; four experts take 1.27 and 1.38 s). At this capacity MICE is more accurate on the aligned grid, by 6.7 points over the archive and 7.5 over the snapshot pool (95% intervals $[+5.4, +8.7]$ and $[+6.2, +9.5]$, all eight streams), and on a source-balanced subset of nine real streams (Airlines 4d, CoverType 4d, PokerHand 4d, INSECTS abr., Rialto, Electricity, Weather, Phishing and Spam, full streams), by 0.35 and 0.27 points ($[+0.26, +0.44]$ and $[+0.16, +0.36]$; above them on seven and six of the nine). Both capped pools also stay within the time of MICE on each of the four grid streams (0.78 and 0.80 s on average). On the shifted grid with fixed segmentation, MICE is 1.45 points above the archive ($[+1.03, +2.06]$), which at size 12 is 2.88 points above MICE over all batches (2.99 on the matched batches of Table 7), and

TABLE 8
Detected segmentation on 72 fresh grid streams (seeds 40 to 43, three random boundary offsets each; 12 streams per cell, 24 per regime): differences in points. Det.: MICE with detected segments; Fixed: MICE with fixed 500-row segments; Archive: unmerged pool with the same detected segments; Rec.: first five batches of each recurring block. Pure: share of segment closes whose receiving context holds at least 95% rows of one concept; Purity: mean majority share; Found: detected changes over true changes; False: cuts away from any boundary.

		over the whole stream				Rec.					
Blocks	cent.	Det.−Reset	Fixed−Reset	Det.−Fixed	Det.−Archive	Det.−Reset	Fixed−Reset	Pure, %	Purity, %	Found	False
500	30	+2.15	-1.51	+3.66	-0.24	+5.1	-0.7	68	95.6	96/96	0
500	100	+8.20	+2.84	+5.36	+0.46	+16.0	+7.5	71	95.8	91/96	0
all 500		+5.17	+0.66	+4.51	+0.11	+10.5	+3.4	69	95.7	187/192	0
2 000	30	+0.53	-0.07	+0.60	-0.19	+4.3	+1.1	95	99.0	96/96	0
2 000	100	+2.43	+1.55	+0.88	+0.38	+15.1	+12.6	90	98.6	96/96	0
all 2 000		+1.48	+0.74	+0.74	+0.10	+9.7	+6.8	92	98.8	192/192	0
500–2 500	30	+0.80	+0.03	+0.77	-0.15	+4.3	+1.4	86	98.2	96/96	0
500–2 500	100	+3.01	+1.27	+1.75	+0.41	+15.8	+10.1	90	98.5	95/96	1
all 500–2 500		+1.91	+0.65	+1.26	+0.13	+10.1	+5.7	88	98.3	191/192	1
All		+2.85	+0.68	+2.17	+0.11	+10.1	+5.3	88	97.6	570/576	1

TABLE 9
Detected against fixed segmentation on every stream group with both (points): MICE with detected segments (Det.) and with fixed segments (Fixed) minus the reference policy, their difference with a 95% moving-block bootstrap interval of the group mean and the number of streams where it is positive, and Det. minus the archive with the same detected segments.

Streams	n	Ref.	Det.	Fixed	Det.−Fixed	95% CI	>0	Det.−Arch.
Aligned grid	8	Reset	+3.13	+3.09	+0.04	[-0.03, +0.11]	6/8	+0.35
Shifted grid	8	Reset	+3.07	-3.47	+6.55	[+5.09, +8.54]	8/8	+0.26
Misaligned	12	Reset	+2.21	+0.47	+1.74	[+1.48, +2.39]	12/12	+0.27
Fresh, random phase	72	Reset	+2.85	+0.68	+2.17	[+1.89, +2.50]	68/72	+0.11
Real	29	FIFO	+0.58	+0.57	+0.01	[-0.02, +0.05]	19/29	+0.08

TABLE 10
Pool controls on the aligned grid (seeds 20, 21): accuracy (%), MICE minus archive and minus snapshot pool (points) with 95% moving-block bootstrap intervals, TFM calls per batch (measured by the call counter of each run; MICE including its discrepancy calls / archive / snapshot), and stored rows per batch in thousands (MICE: bound from the 500/1000-row caps; archive and snapshot: reconstructed exactly).

cent.	block	seed	MICE	Archive	Snapshot	−Arch.	95% CI	−Snap.	95% CI	calls (arch.)	(snap.)	rows, k (arch.)	(snap.)
5	500	20	87.2	87.2	83.9	+0.00	[+0.00, +0.00]	+3.25	[+0.5, +5.9]	6.2	7.1	7.1 1.2–2.4	2.0 3.6
5	500	21	90.0	90.0	86.4	+0.00	[+0.00, +0.00]	+3.57	[+0.5, +6.6]	6.2	7.1	7.1 1.2–2.4	2.0 3.6
5	2000	20	96.5	96.5	96.3	-0.03	[-0.07, +0.01]	+0.25	[-0.1, +0.8]	6.5	12.9	12.9 1.3–2.6	4.9 9.7
5	2000	21	97.1	97.1	97.1	+0.00	[-0.06, +0.08]	+0.00	[-0.1, +0.1]	6.5	12.9	12.9 1.3–2.6	4.9 9.7
30	500	20	81.4	81.4	76.4	+0.02	[-0.02, +0.30]	+4.98	[+1.3, +8.7]	6.2	7.1	7.1 1.2–2.4	2.0 3.6
30	500	21	82.9	82.9	77.4	+0.02	[-0.07, +0.07]	+5.57	[+1.2, +9.5]	6.2	7.1	7.1 1.2–2.4	2.0 3.6
30	2000	20	93.4	93.4	93.6	+0.06	[-0.06, +0.18]	-0.13	[-0.3, +0.0]	6.5	12.9	12.9 1.3–2.6	4.9 9.7
30	2000	21	94.5	94.5	94.5	-0.02	[-0.12, +0.07]	+0.00	[-0.1, +0.2]	6.5	12.9	12.9 1.3–2.6	4.9 9.7
100	500	20	73.5	73.2	66.2	+0.39	[-0.23, +0.82]	+7.32	[+1.8, +11.8]	6.2	7.1	7.1 1.2–2.4	2.0 3.6
100	500	21	70.3	69.2	60.4	+1.07	[+0.14, +1.77]	+9.91	[+2.4, +16.0]	6.2	7.1	7.1 1.2–2.4	2.0 3.6
100	2000	20	86.3	85.9	87.2	+0.37	[+0.01, +0.77]	-0.91	[-1.4, -0.4]	6.5	12.9	12.9 1.3–2.6	4.9 9.7
100	2000	21	85.2	84.7	85.9	+0.45	[+0.21, +0.75]	-0.73	[-1.1, -0.3]	6.5	12.9	12.9 1.3–2.6	4.9 9.7
30/100			83.4	83.1	80.2	+0.30	[+0.12, +0.41]	+3.25	[+1.8, +4.4]	6.3	10.0	10.0 1.2–2.5	3.5 6.7

level with the snapshot pool (+0.00, [−0.64, +0.47], above it on four of the eight streams): the latest rows do not depend on where a segment starts, so the snapshot pool loses nothing when the boundaries move inside the segments, while merged fixed segments do (Section 5.2). With four experts, slower than MICE, both pools are level with it on the shifted grid (−0.01 and −0.09) and remain below it on the aligned grid (by 2.11 and 6.31 points) and on the real subset (by 0.31 and 0.20; above them on six and four of the nine). Organising past contexts by discrepancy thus matches, with 1.8 stored experts on average on the benchmark prefixes, a twelve-expert archive on the aligned grid and both twelve-expert pools on the real streams; where fixed segments mix concepts, unmerged pools are more robust: a snapshot pool sized to the same prefix budget is as accurate, as are both pools of size 4, and an archive of size 12 is more accurate. Detected segments remove this case: on the shifted grid MICE is then 0.26 points above an archive of size 12 with the same segments (Table 9) and 3.42 points above a snapshot pool of size 12 (Table 5).

Cost and a larger window. The pool raises the cost per batch, and the baselines above use contexts of at most 1000 rows. In an isolated benchmark (one process on an NVIDIA RTX 5000 Ada with 32 GB under WSL2, TabPFN v2 with four estimators, GPU memory capped at 28 GB; the first 300 batches of an Airlines, a CoverType and an INSECTS segment; three repetitions), MICE takes 1.07 s per batch (0.91 on batches without a segment close, 1.69 with one), against 0.22 s for FIFO with 1000 rows, 0.42 s with 3000 rows and 1.91 s with 13 400 rows, the capacity MICE may retain (Table 13). A 3000-row window, about 2.5 times cheaper on these prefixes, separates two regimes on the 29 real streams (Table 15 in Appendix E). MICE is ahead by +0.31 points on average (95% interval [+0.09, +0.49]) because of the strongly drifting streams: four INSECTS streams by +3.3 to +5.5 points and Rialto by +6.9. On Airlines (−1.1 points on average over its segments), PokerHand (−1.5), Phishing (−1.4) and Weather (−0.5) the larger window is more accurate, and overall it is ahead on 21 of the 29 streams, with a paired interval below zero on 14. The benchmark prefixes show the same split: larger windows win on Airlines and CoverType and lose up to 14.6 points on INSECTS. The pattern is consistent with the two regimes of Theorem 1: where relearning a concept is cheap, a larger window uses the budget better, and recall pays where concepts change sharply or recur.

Discrepancy. On 144 pairs of grid segments, the recall regret and the excess-risk discrepancy separate "different concept" from "same concept" with AUROC 1.00, and the R-divergence estimate with 0.85 (0.42 on the hardest cell). This is an empirical comparison of the estimators as implemented (accuracy-based, finite contexts), consistent with the direction of Proposition 1 but not a consequence of it, since the proposition concerns population risks of exact concept predictors; on pairs of the same concept the estimated excess is negative, down to −0.2 for 100 centroids, the pooling gain.

Beyond accuracy. Table 14 adds the macro-F1 score, the one-vs-rest area under the receiver operating characteristic curve (AUC), the expected calibration error (ECE) and the

TABLE 11
Pool controls on the 29 real streams: MICE accuracy (%), difference to the archive and to the snapshot pool (points) with one-sided 95% block-bootstrap lower bounds (LB), active experts per batch (MICE / archive; the snapshot pool has the archive's), TFM calls per batch (measured by the call counter of each run; MICE including its discrepancy calls / archive / snapshot), and stored rows per batch in thousands (MICE: bound from the 500/1000-row caps; archive and snapshot: reconstructed exactly).

Stream	batches	MICE	−Arch.	LB	−Snap.	LB	experts	(arch.)	calls	(arch.)	(snap.)	rows, k	(arch.)	(snap.)
Electricity	452	81.22	-0.10	-0.31	-0.02	-0.24	11.2	14.2	13.2	14.2	14.2	4.1–8.2	5.6	11.1
INSECTS abr.	527	77.85	+0.15	-0.07	-0.02	-0.26	8.1	14.3	9.4	13.9	14.1	2.6–5.1	5.6	11.2
INSECTS grad.	240	80.42	-0.17	-0.34	-0.35	-0.64	7.1	13.4	8.0	12.4	12.7	2.1–4.1	5.2	10.3
INSECTS inc.	569	67.77	-0.31	-0.48	-0.49	-0.67	7.0	14.3	8.2	14.3	14.3	2.0–4.0	5.7	11.3
INSECTS inc.-rec.	798	80.80	+0.01	-0.10	-0.09	-0.23	12.1	14.5	13.6	13.6	13.7	4.6–9.1	5.8	11.5
CoverType	1499	95.05	+0.18	+0.04	+0.05	-0.16	14.7	14.7	17.4	14.6	14.6	5.9–11.7	5.9	11.7
INSECTS abr.-imb.	1499	79.81	-0.32	-0.38	-0.52	-0.60	5.8	14.7	6.7	14.7	14.7	1.4–2.8	5.9	11.7
INSECTS grad.-imb.	1432	81.89	-0.13	-0.20	-0.30	-0.38	8.0	14.7	9.4	14.7	14.7	2.5–5.0	5.9	11.7
INSECTS inc.-abr.	798	78.56	+0.41	+0.24	+0.30	+0.12	11.6	14.5	12.4	13.1	13.3	4.3–8.6	5.8	11.5
Airlines	999	69.06	+0.25	+0.10	+0.07	-0.06	6.6	14.6	7.7	14.6	14.6	1.8–3.6	5.8	11.6
Phishing	109	94.60	-0.01	-0.09	-0.14	-0.28	4.5	11.5	5.1	11.5	11.5	0.7–1.5	4.3	8.3
PokerHand	999	92.97	+0.14	-0.25	+0.14	-0.35	12.4	14.6	14.7	14.6	14.6	4.7–9.4	5.8	11.6
Rialto	821	82.19	+0.94	+0.60	+0.36	-0.03	13.7	14.5	16.2	14.5	14.5	5.3–10.7	5.8	11.5
Spam	61	94.25	-0.28	-0.70	-0.33	-0.77	6.1	8.8	6.1	8.0	8.5	1.5–3.1	2.9	5.3
Weather	180	81.07	-0.04	-0.25	-0.19	-0.45	4.0	12.9	4.6	12.9	12.9	0.5–1.0	5.0	9.7
Airlines 3b	999	70.48	+0.15	+0.06	-0.00	-0.08	5.6	14.6	6.5	14.6	14.6	1.3–2.6	5.8	11.6
Airlines 3c	999	65.23	+0.03	-0.07	-0.14	-0.27	7.2	14.6	8.5	14.6	14.6	2.1–4.2	5.8	11.6
CoverType 3b	999	96.58	+0.09	+0.05	+0.07	+0.04	13.4	14.6	15.8	14.6	14.6	5.2–10.4	5.8	11.6
CoverType 3c	999	95.72	+0.15	+0.12	+0.10	+0.07	4.0	14.6	4.6	14.6	14.6	0.5–1.0	5.8	11.6
INSECTS 3b	999	81.67	-0.21	-0.28	-0.37	-0.45	4.7	14.6	5.5	14.6	14.6	0.9–1.7	5.8	11.6
INSECTS 3c	999	85.66	-0.18	-0.26	-0.29	-0.38	6.6	14.6	7.7	14.6	14.6	1.8–3.6	5.8	11.6
PokerHand 3b	999	93.98	+0.71	+0.42	+0.40	+0.13	10.7	14.6	12.6	14.6	14.6	3.9–7.7	5.8	11.6
PokerHand 3c	999	92.16	+0.08	-0.27	-0.36	-0.83	11.7	14.6	13.8	14.6	14.6	4.3–8.7	5.8	11.6
Airlines 4d	999	65.99	+0.08	-0.04	-0.22	-0.34	6.5	14.6	7.6	14.6	14.6	1.8–3.5	5.8	11.6
Airlines 4e	999	65.59	+0.14	-0.00	-0.14	-0.28	7.5	14.6	8.8	14.6	14.6	2.3–4.5	5.8	11.6
CoverType 4d	999	96.24	+0.14	+0.10	+0.14	+0.10	4.1	14.6	4.6	14.6	14.6	0.5–1.1	5.8	11.6
CoverType 4e	999	97.19	+0.12	+0.08	+0.11	+0.08	4.3	14.6	4.9	14.6	14.6	0.7–1.3	5.8	11.6
PokerHand 4d	999	94.49	+0.34	+0.06	+0.19	-0.10	11.1	14.6	13.1	14.6	14.6	4.0–8.1	5.8	11.6
PokerHand 4e	999	92.34	-0.21	-0.62	-0.19	-0.63	12.1	14.6	14.3	14.6	14.6	4.6–9.1	5.8	11.6
Mean		83.82	+0.07		-0.08		8.4	14.2	9.7	14.0	14.1	2.7–5.4	5.6	11.1

half Brier score, computed from the stored predictions of every method. On the real streams the picture of the accuracy carries over for F1 and the Brier score: MICE has the highest F1 and a lower Brier score than every baseline; against FIFO its F1 is higher on 24 of 29 streams (+0.8 points on average), its AUC on 25 and its Brier score lower on 26, and against reset on 25, 28 and 28. The exception is the AUC of the dual memory: LTM has the highest mean AUC (91.0 against 90.5), all of it from the five PokerHand segments (up to 7.3 points), where its class-balanced long-term buffer keeps rows of the rare classes that a window loses; on the same segments its accuracy is 2.5 to 3.8 points below MICE, and on the other 24 streams the AUC of MICE is 0.2 points higher in the median. On the grid the step $\eta_2 = \frac{1}{2}$ has a price in the quality of the probabilities: MICE is 0.2 points below the step of 10 in accuracy but 3.1 points in ECE and 1.2 in the Brier score, because a small step keeps weight on the default window, which is poorly calibrated under conflicting concepts (ECE 16.7); its ECE is also above that of reset (7.3), while its Brier score is lower (12.7 against 13.1). The step that Theorem 2 covers is thus the conservative choice: it buys the guarantee with calibration in the recall regime.

6 Related Work

Concept drift and recurring concepts. Drift detection and adaptation are surveyed by Gama et al. [1] and Lu et al. [2]. Recurring concepts are handled by storing past models and reusing them, selected by statistical tests [3], by meta-learners of each model's domain [4] or by weighted ensembles of batch learners [21]. MOOE [5] combines offline experts of past intervals with an online expert under a meta expert with regret guarantees; MICE keeps its decomposition and replaces trained experts by stored contexts, the cumulative meta expert by two time scales, and the generic regret by a quantity measured from the learning curve. Model-oriented discrepancy. R-divergence [15] and I-Div [28] measure distribution discrepancy through a learned hypothesis; we identify the blind spot of the former under conflicting concepts and use an excess-risk form to decide which contexts to pool. TFMs on streams. Lourenço et al. [10] show that a TFM with a short- and long-term FIFO memory outperforms incremental ensembles; Drift-Resilient TabPFN [11] retrains the prior to extrapolate temporal trends. Neither models recurrence. Concept drift has likewise been redefined for other new model families, such as multi-modal large language models [29, 30]. Online aggregation. Exponential weights with exp-concave losses and their logarithmic regret are classical [31, 32]; Theorem 2 is their discounted, any-time form for two in-context experts.

7 Conclusion

For an in-context learner, a past concept is recalled without retraining, and the value of recalling it is the area between the learning curve and the accuracy of the stored context. MICE

TABLE 12
MICE against the archive and snapshot pools at pool sizes k of 2, 4 and 12. Mean stored experts, TFM calls and stored rows (thousands) per batch, seconds per batch and peak reserved GPU memory (GB) from the isolated benchmark on the three real prefixes (mean over prefixes of the mean over three repetitions; one repetition for $k = 12$); seconds per batch on four grid streams; accuracy difference MICE minus control (points) on the aligned grid (8 streams), the shifted grid (8) and the real subset (9), with 95% block-bootstrap intervals of the group mean. $k = 2$ is the capacity chosen from the timings, $k = 4$ a supplementary configuration; $k = 12$ differences are on the same streams, without intervals.

	resources						MICE minus control, points		
Pool	experts	calls	rows, k	s, real	s, grid	GB	aligned	shifted	real
MICE	1.8	5.5	1.8	1.11	1.05	0.99			
Archive, $k = 2$	2.0	4.9	1.0	0.93	0.78	0.76	+6.72 [+5.42, +8.69]	+1.45 [+1.03, +2.06]	+0.35 [+0.26, +0.44]
Snapshot, $k = 2$	2.0	4.9	1.9	0.98	0.80	0.76	+7.49 [+6.18, +9.51]	+0.00 [−0.64, +0.47]	+0.27 [+0.16, +0.36]
Archive, $k = 4$	3.8	6.7	1.9	1.27	–	0.76	+2.11 [+1.53, +2.74]	−0.01 [−1.13, +0.62]	+0.31 [+0.20, +0.41]
Snapshot, $k = 4$	3.8	6.8	3.8	1.38	–	0.76	+6.31 [+4.65, +8.09]	−0.09 [−1.19, +0.49]	+0.20 [+0.08, +0.30]
Archive, $k = 12$	10.7	13.6	5.4	2.49	1.58	0.76	+0.30	−2.88	+0.13
Snapshot, $k = 12$	10.7	13.6	10.6	2.84	1.63	0.76	+3.25	–	−0.03

TABLE 13
Isolated resource benchmark (registered; one process on an RTX 5000 Ada, GPU memory capped at 28 GB; first 300 batches of three real segments; mean over the three prefixes of the mean over three repetitions). Seconds per batch (MICE including its two-level rule and segment processing), rows per second, peak allocated and reserved GPU memory and peak process memory (GB), and accuracy (%) on the same prefixes.

	seconds per batch					memory, GB			accuracy		
Policy	mean	median	p95	max	rows/s	alloc.	reserved	process	Airlines	CoverType	INSECTS
MICE	1.07	0.95	2.12	2.66	99	0.2	1.0	1.8	65.3	95.4	78.1
FIFO 1000	0.22	0.21	0.28	1.48	471	0.2	0.8	1.7	65.3	95.4	76.9
FIFO 1500	0.26	0.25	0.32	1.44	403	0.2	1.4	1.7	66.0	95.7	76.5
FIFO 2000	0.31	0.30	0.38	1.51	354	0.3	2.3	1.7	66.3	95.8	75.5
FIFO 3000 (M^*)	0.42	0.42	0.49	1.43	282	0.4	4.7	1.7	66.6	95.7	73.4
FIFO 5000	0.67	0.71	0.78	1.46	185	0.6	12.1	1.7	66.8	95.7	69.8
FIFO 13 400	1.91	2.49	2.59	3.12	71	1.4	26.4	1.7	66.8	95.6	63.5
Window ens.	0.53	0.50	0.65	1.80	192	0.2	0.8	1.7	64.1	95.3	77.7

TABLE 14
Metrics beyond accuracy (%, mean over streams): macro-F1, one-vs-rest ROC AUC, expected calibration error (15 bins) and half Brier score, on the 29 real streams and the 24 grid streams. Computed from stored predictions; the best value per column is in bold.

	Real streams					Controlled grid				
Method	Acc. ↑	F1 ↑	AUC ↑	ECE ↓	Brier ↓	Acc. ↑	F1 ↑	AUC ↑	ECE ↓	Brier ↓
FIFO	83.3	73.5	89.9	1.7	11.6	61.9	60.9	83.7	16.7	25.1
LTM	82.6	72.1	91.0	1.5	11.9	62.2	61.4	84.8	16.0	24.5
Reset	81.5	71.3	88.5	2.6	12.9	84.1	83.8	91.4	7.3	13.1
Window ensemble	83.0	73.0	89.5	1.2	11.7	81.6	81.0	90.8	8.7	14.9
Fast level only	83.3	73.4	90.3	1.5	11.5	86.8	86.5	93.6	7.1	11.4
MICE, $\eta_2 = 10$	83.8	74.2	90.5	1.1	11.2	86.6	86.3	93.6	7.0	11.5
MICE	83.8	74.3	90.5	1.3	11.2	86.4	86.0	92.3	10.1	12.7

turns this into a stream learner: past contexts are experts next to fast windows, a fast meta expert identifies a recurring concept within one batch, and a slow meta expert bounds, at any time and for any stream, what following the mixture costs against the plain window. In the population limit a stored context of deterministic concepts with a shared input distribution recalls the concept that holds the majority of its rows. Segments that start at detected changes kept a majority at every close we measured, wherever the concept boundaries fell, on grid concepts that are not deterministic, so this share describes their mixing; they are not pure, which the recall theorem assumes. The experiments support both regimes of the theory. Where concepts are costly to learn, recall gains up to 8.5 points over reset detection, in the order the learning curves predict; fixed segments lose this gain when boundaries fall inside them, and detected segments recover it on fresh streams with random boundary phases; where relearning is cheap there is little to recall. Merging by discrepancy keeps the pool small: with detected segments it holds a third of the experts of an unmerged archive, is 0.11 points more accurate and takes 0.41 times its time per batch on three real prefixes, and it is more accurate than a snapshot pool of the latest rows with the same capacity, except on 2 000-row blocks, where they are level; with fixed segments and sizes chosen to fit its time per batch on three real prefixes, unmerged pools are less accurate, except a snapshot of the latest rows on the shifted

grid, which matches MICE. MICE is never more than 0.25 points below the plain window on the 29 real streams, and a larger window can be the better use of the budget.

Limitations. With fixed 500-row segments, both tests with boundaries inside segments failed their registered criteria, MICE fell below reset on most shifted streams, and an archive of unmerged segments was more accurate. Detected segments repair this on the grid, but their stored contexts are not pure: at least 95% rows of one concept in 90% of the closes was not reached (88%, 87% on the confirmation streams, 69% with 500-row blocks), because a boundary inside a batch leaves up to 99 rows of the neighbouring concept in a segment, so Theorem 1 is not verified for these contexts and Proposition 2 covers only the population limit. The detection guarantee assumes window errors that do not rise between changes; on real streams the detector declares up to 180 changes on one stream, and there the slow level, not the detector, keeps MICE safe. The generator streams, the confirmatory real sets, the second backbone seed, TabICL, the pool controls and the timings were obtained with fixed segmentation, except the resource and snapshot comparison of Table 5. At the same pool size, merging by discrepancy did not improve accuracy over an unmerged archive by the registered margin with fixed segments; its benefit is cost: with sizes chosen to fit the time per batch of MICE on three real prefixes, both controls were less accurate except the snapshot pool on the shifted grid, which matched MICE, as did both controls with four experts, which are slower. On real streams the gains are large only on sharply drifting streams (INSECTS, Rialto); a 3000-row window, about 2.5 times cheaper on the benchmark prefixes, was more accurate on 21 of the 29 streams, and costs were measured on three 300-batch prefixes on one GPU only. The confirmatory real segments come from three streams whose earlier segments were used in development, and most streams have one run. Theorem 2 bounds the half Brier score rather than the accuracy, and its step costs calibration when concepts recur (Table 14); Propositions 1 and 2 hold under Assumption 1, Proposition 3 under Assumption 2, and Theorem 1 under Assumption 3.

References

[1] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," ACMComput.Surv., vol. 46, no. 4, pp. 44:1–44:37, 2014.

[2] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," IEEETrans.Knowl.DataEng., vol. 31, no. 12, pp. 2346–2363, 2019.

[3] P. M. Gonçalves, Jr. and R. S. M. de Barros, "RCD: A recurring concept drift framework," Pattern Recognit. Lett., vol. 34, pp. 1018–1025, 2013.

[4] J. Gama and P. Kosina, "Recurrent concepts in data streams classification," Knowl. Inf. Syst., vol. 40, 2014.

[5] Z. Zhao, L. Cao, and Y. Wan, "Mixture of online and offline experts for non-stationary time series," in Proc.AAAI, 2025.

[6] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter, "Accurate predictions on small data with a tabular foundation model," Nature, vol. 637, no. 8045, pp. 319–326, 2025.

[7] J. Qu, D. Holzmüller, G. Varoquaux, and M. L. Morvan, "TabICL: A tabular foundation model for in-context learning on large data," in Proc.ICML, 2025, pp. 50 817–50 847.

[8] S. Müller, N. Hollmann, S. Pineda-Arango, J. Grabocka, and F. Hutter, "Transformers can do Bayesian inference," in Proc.ICLR, 2022.

[9] T. Nagler, "Statistical foundations of prior-data fitted networks," in Proc.ICML, 2023, pp. 25 660–25 676.

[10] A. Lourenço, J. Gama, E. P. Xing, and G. Marreiros, "In-context learning of evolving data streams with tabular foundational models," in Proc.ACMSIGKDD, 2026.

[11] K. Helli, D. Schnurr, N. Hollmann, S. Müller, and F. Hutter, "Drift-resilient TabPFN: In-context learning temporal distribution shifts on tabular data," in Proc.NeurIPS, 2024, pp. 98 742–98 781.

[12] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in Advances in Artificial Intelligence – SBIA 2004, ser. Lecture Notes in Computer Science, vol. 3171. Springer, 2004, pp. 286–295.

[13] J. C. Schlimmer and R. H. G. Jr., "Incremental learning from noisy data," Mach.Learn., vol. 1, no. 3, pp. 317–354, 1986.

[14] R. Agrawal, T. Imielinski, and A. Swami, "Database mining: A performance perspective," IEEETrans.Knowl.DataEng., vol. 5, no. 6, pp. 914–925, 1993.

[15] Z. Zhao and L. Cao, "R-divergence for estimating model-oriented distribution discrepancy," in Proc.NeurIPS, 2023.

[16] H. M. Gomes, A. Bifet, J. Read, J. P. Barddal, F. Enembreck, B. Pfahringer, G. Holmes, and T. Abdessalem, "Adaptive random forests for evolving data stream classification," Mach.Learn., vol. 106, no. 9–10, pp. 1469–1495, 2017.

[17] H. M. Gomes, J. Read, and A. Bifet, "Streaming random patches for evolving data stream classification," in Proc. IEEE International Conference on Data Mining (ICDM), 2019, pp. 240–249.

[18] A. Bifet and R. Gavaldà, "Adaptive learning from evolving data streams," in Proc. International Symposium on Intelligent Data Analysis (IDA), 2009, pp. 249–260.

[19] A. Bifet, G. Holmes, and B. Pfahringer, "Leveraging bagging for evolving data streams," in Proc. ECML PKDD, 2010, pp. 135–150.

[20] A. Bifet, G. Holmes, B. Pfahringer, R. Kirkby, and R. Gavaldà, "New ensemble methods for evolving data streams," in Proc.ACMSIGKDD, 2009, pp. 139–148.

[21] R. Elwell and R. Polikar, "Incremental learning of concept drift in nonstationary environments," IEEE Trans. Neural Networks, vol. 22, no. 10, pp. 1517–1531, 2011.

[22] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem, and A. Bifet, "River: Machine learning for streaming data in Python," J. Mach. Learn. Res., vol. 22, no. 110, 2021.

[23] W. N. Street and Y. Kim, "A streaming ensemble algorithm (SEA) for large-scale classification," in Proc.ACMSIGKDD, 2001, pp. 377–382.

[24] G. Hulten, L. Spencer, and P. Domingos, "Mining time-changing data streams," in Proc.ACMSIGKDD, 2001, pp. 97–106.
[25] V. M. A. Souza, D. M. dos Reis, A. G. Maletzke, and G. E. A. P. A. Batista, "Challenges in benchmarking stream learning algorithms with real-world data," Data Mining and Knowledge Discovery, vol. 34, no. 6, pp. 1805–1858, 2020.
[26] V. Losing, B. Hammer, and H. Wersing, "KNN classifier with self adjusting memory for heterogeneous concept drift," in Proc. IEEE ICDM, 2016, pp. 291–300.
[27] I. Katakis, G. Tsoumakas, and I. Vlahavas, "Tracking recurring contexts using ensemble classifiers: An application to email filtering," Knowl. Inf. Syst., vol. 22, no. 3, pp. 371–391, 2010.
[28] Z. Zhao, L. Cao, X. Fan, and W.-S. Zheng, "Revealing distribution discrepancy by sampling transfer in unlabeled data," in Proc.NeurIPS, 2024.
[29] X. Yang, J. Lu, and E. Yu, "Adapting multi-modal large language model to concept drift from pre-training onwards," in Proc.ICLR, 2025.
[30] X. Yang, E. Yu, W. Duan, and J. Lu, "Turning drift into constraint: Robust reasoning alignment in non-stationary multi-stream environments," in Proc.ICML, 2026.
[31] N. Cesa-Bianchi and G. Lugosi, Prediction, Learning, and Games. Cambridge University Press, 2006.
[32] E. Hazan, A. Agarwal, and S. Kale, "Logarithmic regret algorithms for online convex optimization," Mach.Learn., vol. 69, pp. 169–192, 2007.

Appendix

.1 Proof of Proposition 1

Proof. For the log-loss and any conditional h,

$$\epsilon_P(h) = \mathbb{E}_{P_X}\big[H(p_{\mathbf{x}}) + \mathrm{KL}(p_{\mathbf{x}} \| h_{\mathbf{x}})\big],$$

so that $\epsilon_P(h_P) = \mathbb{E}_{P_X} H(p_{\mathbf{x}})$ and $\epsilon_P(h_U) - \epsilon_P(h_P) = \mathbb{E}_{P_X}\mathrm{KL}(p_{\mathbf{x}} \| u_{\mathbf{x}})$, and likewise for Q.

(i) By the chain rule, $\mathrm{KL}(P\|U) = \mathrm{KL}(P_X\|U_X) + \mathbb{E}_{P_X}\mathrm{KL}(p_{\mathbf{x}}\|u_{\mathbf{x}})$, and the same holds for Q. Averaging the two identities gives

$$\mathrm{JS}(P, Q) = \mathrm{JS}(P_X, Q_X) + \mathrm{D}_E(P, Q),$$

and $\mathrm{D}_E \geq 0$ as a mean of KL terms. If $P_X = Q_X$, then $\alpha \equiv \frac{1}{2}$, $u_{\mathbf{x}} = \frac{1}{2}(p_{\mathbf{x}} + q_{\mathbf{x}})$ and $\mathrm{D}_E = \mathbb{E}_{\mathbf{x}}\mathrm{JS}(p_{\mathbf{x}}, q_{\mathbf{x}})$, which vanishes if and only if $p_{\mathbf{x}} = q_{\mathbf{x}}$ almost surely.

(ii) By the display above,

$$\mathrm{D}_R(P\|Q) = \Big|\mathbb{E}_{P_X}\big[H(p_{\mathbf{x}}) + \mathrm{KL}(p_{\mathbf{x}}\|u_{\mathbf{x}})\big] - \mathbb{E}_{Q_X}\big[H(q_{\mathbf{x}}) + \mathrm{KL}(q_{\mathbf{x}}\|u_{\mathbf{x}})\big]\Big|.$$

Deterministic concepts have $H(p_{\mathbf{x}}) = H(q_{\mathbf{x}}) = 0$, and $u_{\mathbf{x}}$ puts mass $\frac{1}{2}$ on each of $g_P(\mathbf{x})$ and $g_Q(\mathbf{x})$ where they differ and mass 1 where they agree. Hence $\mathrm{KL}(p_{\mathbf{x}}\|u_{\mathbf{x}}) = \mathrm{KL}(q_{\mathbf{x}}\|u_{\mathbf{x}}) = \ln 2 \cdot \mathbf{1}[g_P(\mathbf{x}) \neq g_Q(\mathbf{x})]$; with $P_X = Q_X$ the two expectations cancel, and $\mathrm{D}_E = \rho \ln 2$.

(iii) The predictor h_P errs on Q exactly where the concepts disagree and h_Q nowhere, so $\mathrm{D}_T(Q \,|\, P) = \rho$ whatever the tie rule of h_U. Where the concepts agree, $u_{\mathbf{x}}$ has a unique maximiser and h_U errs on neither concept. Where they disagree, $u_{\mathbf{x}}$ puts $\frac{1}{2}$ on $g_P(\mathbf{x})$ and $\frac{1}{2}$ on $g_Q(\mathbf{x})$. With uniformly random ties ξ, h_U^ξ predicts each of the two with probability $\frac{1}{2}$, so $\mathbb{E}_\xi \epsilon_P(h_U^\xi) = \mathbb{E}_\xi \epsilon_Q(h_U^\xi) = \rho/2$ and $\mathrm{D}_R = 0$ (this concerns the averaged risks, not the expected absolute difference for one sampled predictor). With a deterministic rule c, h_U errs on P where $c(\mathbf{x}) \neq g_P(\mathbf{x})$ and on Q where $c(\mathbf{x}) \neq g_Q(\mathbf{x})$, restricted to the disagreement region, which gives the stated expression; it is at most ρ because the two indicators differ only there, it is zero for example when c agrees with each concept on half of the disagreement mass, and it equals ρ for example when that region is a single atom. □

.2 Proofs of Propositions 2 and 3 and Corollary 1

Proposition 2. With $P_X = Q_X$ the conditional of $\pi P + (1-\pi)Q$ is $\pi\,\mathbf{1}[y = g_P(\mathbf{x})] + (1-\pi)\,\mathbf{1}[y = g_Q(\mathbf{x})]$. Where $g_P(\mathbf{x}) = g_Q(\mathbf{x})$ its mode is $g_P(\mathbf{x})$; where they differ the mode is $g_P(\mathbf{x})$ if $\pi > \frac{1}{2}$ and $g_Q(\mathbf{x})$ if $\pi < \frac{1}{2}$. The 0-1 risk on P is therefore 0 or ρ, and $\epsilon_P(h_P) = 0$, which gives the regret.

Proposition 3. Fix a candidate b and a number $m \in \{2, \ldots, R_{\mathrm{cp}}\}$ of compared batches before it, and write the statistic of (4) without the margin as $Z_m = \frac{1}{n_2}\sum_{\mathrm{post}} \mathbf{1}[\mathrm{error}] - \frac{1}{n_1}\sum_{\mathrm{pre}} \mathbf{1}[\mathrm{error}]$ over the rows of the batches $b-m, \ldots, b-1$ (pre, $n_1 = mB$) and $b, \ldots, t-1$ (post), and $\bar{Z}_m$ for the same sum of conditional error rates. For fixed b and m these rows and the weights $1/n_1$ and $1/n_2$ do not depend on the data. By Assumption 2 the error indicators minus their conditional rates, taken in row order, form a martingale difference sequence whose terms lie in intervals of length $1/n_2$ (post) and $1/n_1$ (pre). By the Azuma–Hoeffding inequality, $\Pr[Z_m - \bar{Z}_m \geq \lambda] \leq \exp\big(-2\lambda^2/(1/n_1 + 1/n_2)\big)$,

and the same bound holds for $\Pr[Z_m - \bar{Z}_m \le -\lambda]$; at $\lambda = \tau(n_1, n_2)$ both equal δ_{cp}. The reference that (4) uses starts after the previous declared change, which depends on earlier errors, possibly on those of the compared rows; it is one of these $R_{\mathrm{cp}} - 1$ fixed references, and a union bound over m covers this selection. (i) Without a change in the batches $t - H_{\mathrm{cp}} - R_{\mathrm{cp}}, \dots, t-1$, Assumption 2(a) gives $\bar{Z}_m \le 0$ for every b and m, so each excess is positive with probability at most δ_{cp}; a union bound over the at most H_{cp} candidates and $R_{\mathrm{cp}} - 1$ references gives the claim. (ii) On the event that b is a candidate with $m \ge 2$ compared batches, these batches form one of the runs of Assumption 2(b), so $\bar{Z}_m \ge \beta \ge 2\tau(2B, n_2) \ge 2\tau(mB, n_2)$, as τ decreases in n_1; the excess $Z_m - \tau(mB, n_2)$ is then non-positive only if $Z_m - \bar{Z}_m \le -\tau(mB, n_2)$, and a union bound over the $R_{\mathrm{cp}} - 1$ values of m bounds the probability of this event by $(R_{\mathrm{cp}} - 1)\delta_{\mathrm{cp}}$.

Corollary 1. Each end of a closed segment is a declared change, a periodic close inside one concept, or the start of the stream. Without missed boundaries, the rows of other concepts lie between an end and the true boundary nearest to it, at most ν rows at each end, so at most 2ν in all; the majority concept holds more than half of the rows when $2\nu < |S|/2$.

.3 Proofs of Lemma 1 and Proposition 4

Proof of Lemma 1. (i) By Assumption 3,

$$w_{k^\star} = \Big(1 + \sum_{k \ne k^\star} e^{-\eta(\ell_k - \ell_{k^\star})}\Big)^{-1} \ge \big(1 + (K-1)e^{-\eta g}\big)^{-1} = e^{-\kappa}.$$

(ii) For a row with label y and expert predictions p_k,

$$-\ln \sum_k w_k p_k(y) \;\le\; -\ln\big(w_{k^\star} p_{k^\star}(y)\big) \;\le\; -\ln p_{k^\star}(y) + \kappa.$$

(iii) For two classes y_1, y_2,

$$\sum_k w_k\big(p_k(y_1) - p_k(y_2)\big) \;\ge\; w_{k^\star}\big(p_{k^\star}(y_1) - p_{k^\star}(y_2)\big) - (1 - w_{k^\star}),$$

which is positive when the gap of expert $k^\star$ exceeds $(1 - w_{k^\star})/w_{k^\star} \le e^\kappa - 1$. □

Proof of Proposition 4. The first batch of the block is predicted with weights from a batch of the previous concept; its per-row regret is at most c. Every later batch is predicted with weights from a labelled batch of the block, on which $k^\star$ leads by g, so Lemma 1(ii) bounds its per-row regret by $\ln\big(1 + (K-1)e^{-\eta g}\big) \le (K-1)e^{-\eta g}$. Summing over the $J-1$ later batches gives the claim. □

.4 Proofs of Theorem 1 and Corollary 2

Proof of Theorem 1. The first batch of the block is predicted by both learners before any label of the block has arrived; its accuracy difference, divided by J, is $\zeta_{k,c}$ and is at most $1/J$ in absolute value because accuracies lie in $[0, 1]$. Before batch $j+1$ ($j = 1, \dots, J-1$) the relearning context holds $\min(jB, M)$ rows of the block, so its expected accuracy is $a_k(\min(jB, M))$; the stored expert has expected accuracy $a_k(n_c)$. By Lemma 1(iii) the mixture predicts as the better of the two, so its expected accuracy is the larger of the two values, and the difference to relearning is the positive part in (7). Averaging over the J batches gives the area term, and adding the first batch gives $\zeta_{k,c}$. □

Proof of Corollary 2. On the first visit $n_1 = 0$, and $a_k(0) \le a_k(\min(jB, M))$ because a_k is non-decreasing, so every positive part vanishes. Since a_k is non-decreasing and $n_c \le M$, every term of the area term is at most $a_k(M) - a_k(B)$, which is zero for a flat curve, and the terms with $jB \ge M$ vanish. At most M/B of the $J-1$ terms are non-zero (only those with $jB < n_c \le M$), which with $|\zeta_{k,c}| \le 1/J$ gives the bound. □

.5 Proofs of Theorem 2 and Corollary 3

Proof of Theorem 2. Exp-concavity. The half Brier score $\ell(p) = \frac{1}{2}\|p - e_y\|^2$ has gradient $p - e_y$ and Hessian I, and $\|p - e_y\|^2 \le 2$ on the simplex. Hence $\nabla^2 \ell \succeq \eta_2 \nabla\ell\nabla\ell^\top$ for $\eta_2 \le \frac{1}{2}$, that is, ℓ is η_2-exp-concave. The mean over the rows of a batch is η_2-exp-concave as well: its Hessian is the mean Hessian, and $\frac{1}{n}\sum_i g_i g_i^\top \succeq \bar{g}\bar{g}^\top$ for the row gradients g_i with mean $\bar{g}$. Consequently,

$$e^{-\eta_2 \ell_t(p_t)} \;\ge\; \sum_k v_{t,k}\, e^{-\eta_2 \ell_{t,k}}. \tag{12}$$

Potential. Let $W_t = \sum_k e^{-\eta_2 L_{t,k}}$, so that $W_0 = K$. By (8) and (9),

$$W_t = \Big(\sum_k e^{-\eta_2 \gamma L_{t-1,k}}\Big) \sum_k v_{t,k}\, e^{-\eta_2 \ell_{t,k}}.$$

By the power-mean inequality, $\sum_k x_k^\gamma \le K^{1-\gamma}\big(\sum_k x_k\big)^\gamma$ for $x_k \ge 0$ and $\gamma \in (0, 1]$; with $x_k = e^{-\eta_2 L_{t-1,k}}$ and (12),

$$\ln W_t \;\le\; (1-\gamma)\ln K + \gamma \ln W_{t-1} - \eta_2\, \ell_t(p_t).$$

Unrolling. Since $(1-\gamma)\sum_{t \le T} \gamma^{T-t} + \gamma^T = 1$,

$$\ln W_T \;\le\; \ln K - \eta_2 \sum_{t=1}^{T} \gamma^{T-t} \ell_t(p_t).$$

On the other hand $\ln W_T \ge -\eta_2 L_{T,k} = -\eta_2 \sum_{t=1}^T \gamma^{T-t} \ell_{t,k}$ for every k. Combining the two inequalities gives (11). □

Proof of Corollary 3. Insert $K = 2$ and $\eta_2 = \frac{1}{2}$ into (11); the discounted number of batches is $\sum_{t \le T} \gamma^{T-t} \le 1/(1-\gamma)$. □

Development and provenance. Resource benchmark: registered before it was run; the comparison window was fixed beforehand from timings alone as the size among 1500, 2000 and 3000 rows whose cost is within a factor 1.25 of MICE's; none was, so the closest, 3000 rows, was used and the comparison is not cost-matched. The benchmark initially ran without a GPU memory cap; one oversubscribed run was discarded and rerun under a 28 GB cap, recorded before resuming. MICE's expert predictions in the benchmark are bit-identical to those behind every reported number. Costs were measured on the three prefixes only; the 29-stream comparison is an accuracy comparison. Pool controls: the archive and snapshot controls were added in response to review. The archive runs, the grid criterion (MICE above the archive by at least 0.5 points on average over the eight streams with 30 or 100 centroids and on at least six of them; failed) and the real-stream criterion for the archive (mean difference at least -0.1 points; held) were registered before the runs. The grid snapshot pool was

registered as a secondary control without a criterion. The snapshot pool on the real streams and its descriptive threshold (at least -0.1 points; held) were registered after the 29 streams and the MICE results had been examined, so that comparison is a review-added control and not an independent confirmation. The grid intervals (moving-block bootstrap of the aligned per-batch differences with the same resampled indices for both policies of a stream, blocks of 10; the eight-stream aggregate is the mean of independently resampled streams) and the real-stream bounds against the archive are retrospective. MICE's discrepancy calls were reconstructed as calls per batch minus active experts per batch (0.05 to 2.6 per batch; a lower estimate because experts with a single-class context make no TFM call); stored rows of the archive and snapshot pool were reconstructed exactly from their storage rules; MICE's storage is a bound and at the upper cap can exceed the archive on single streams. The two boundary tests (misaligned schedules and shifted boundaries) were added in response to review and registered before they were run. The realigned-segment test (c) was added later, after the two failed, and registered with its three criteria before it was run: MICE above reset on at least 7 of the 8 streams with a mean gain of at least one point, a mean phase-restoration effect of at least 5.0 points, and a mean history effect within one point; all three held. The registered criterion of the misaligned schedules was MICE above reset on at least 10 of the 12 streams with a mean gain of at least one point; that of the shifted boundaries was MICE above reset on at least 7 of the 8 with a mean gain of at least one point. Both failed. The cache holds every expert's prediction for every batch; the fast mixture of batch t is formed from these with weights that depend on the labels of earlier batches only, so replaying the rule over the cache issues the same predictions an online run would. All reported numbers for MICE come from such replays: the method above with $\eta_2 = \frac{1}{2}$ (simulate2(outer='brier', scale=0.5, temper=True) in the released code, frozen on 2026-10-05), and the stated variants with the log-loss or with $\eta_2 = 10$ as replays of the same rule with those parameters. The runs that produced the cache emitted a different, single-level mixture, which we do not report. All constants were set on synthetic development streams; none was tuned on a real stream. Hypotheses, data and decision rules were written down before each test, and every test stream was run once. The real streams were opened in five stages (Table 4). The first three were also used, after their one-time evaluation, to develop the slow level. The third held-out set (eight later segments of four streams) and the grid seeds 20 and 21 were evaluated after the two-level rule was frozen with a step of 10; the step $\frac{1}{2}$ required by Theorem 2 was added to that analysis before any result was computed. The fourth held-out set, six further segments that no method had loaded, was then run once with MICE with fixed segmentation, otherwise as defined in Section 4, as the registered method (Appendix E). The registered predictions used a simplification of the area term: a stored context of M rows on every recurrence, whereas Theorem 1 has $n_c = \min(M, (c-1)L)$, i.e. 500 rows on the second visit when $L = 500$. Recomputed with n_c from the same learning curves, the predictions change only for $L = 500$ (100 centroids: 9.2 instead of 10.4 points; 30 centroids: 1.7 instead of 1.9), the rank correlations are unchanged (0.94 and 0.83) and the magnitudes agree more closely (9.2 versus 8.6). We report the registered values in Table 2 and Figure 2 because they were fixed before the runs. The two seed pairs answer different questions. The curves were measured on the concepts of seeds 10 and 11, so there the prediction is for concepts already sampled, using additional labelled data from them; we also found that this data repeats part of the random draws of the evaluated seed-10/11 streams (Appendix F), and curves remeasured on independent rows give the same rank correlations (0.94 and 0.83) and predictions within 0.6 points. Seeds 20 and 21 share no data with the curves: there the curves of other concept draws from the same generator predict the gains, which is the evidence for transfer to new concepts of the same family, with two data seeds. Every stage was run once when it was opened; the third and fourth held-out sets were opened after the two-level rule was frozen. The oracle test of Table 3 was registered with its seeds and criteria before it was run: the measured area term within 1.0 point of the visit-specific prediction on at least five of the six cells with a mean absolute difference of at most 0.5 points, and a Spearman correlation of at least 0.8; both held. Its TFM uses backbone seed 0, the learning curves seeds 10 and 11. The capacity comparison of Table 12 was added in response to review and registered before it was run: the capacity rule (the largest of 2 and 4 within MICE's amortised time per batch on all three prefixes, from timings alone), the streams, the block bootstrap (blocks of 10 batches on the grid, 20 on real streams) and the rule that a group with a mean difference of at most $+0.1$ points is named in the abstract. The accuracy runs at size 2 were made before the capacity was fixed, to use the GPU while the benchmark waited; no accuracy was read before it was fixed. The runs at size 4 were made afterwards as the supplementary configuration. The real subset was chosen before the runs to balance sources; its $k = 12$ differences come from the 29-stream runs.

All hypotheses below were written to the experiment log before the corresponding streams were run, and every test stream was run once; interrupted runs were resumed from their checkpoints without rerunning a finished cell.

Grid, seeds 10 and 11 (fast level alone). Hypotheses: gain over reset positive on all eight streams with 30 or 100 centroids, larger for shorter blocks and for more centroids; Spearman correlation between predicted and observed gain at least 0.8; the window ensemble below the method on these eight streams. All held (correlation 0.94). An incidental expectation of no gain with 5 centroids was missed on one stream, where the reset baseline detected late.

Generator streams, seeds 10–14, and five real development streams. Non-inferiority to reset on the generator streams ($+0.50$ points, 25 of 30 positive) and on each real stream held, as did the replication of Observation 1. An exploratory expectation that the INSECTS stream with re-occurring drift would gain most was not met (it ranked second).

First held-out set (four streams; fast level alone). Non-inferiority to reset on every stream and highest mean accuracy held; the method was the most accurate on three of four streams. A replication with TabICL held as well.

Second held-out set (six streams; fast level alone). Non-inferiority to reset held. The hypothesis that the fast level alone would be the most accurate method on at least four of six streams was not met (one of six), and on PokerHand it was 2.3 points below FIFO. This result motivated the slow

level; all real streams seen so far count as development data from here on.

Third held-out set (eight segments; two-level rule frozen). Two candidates for the slow level were registered before any result existed: the log-loss with step 1, and the half Brier score with step 10, added after the log-loss failed on a cached synthetic test stream. For both, non-inferiority to FIFO and reset (tolerance 0.5 points) and to the fast level alone held on every segment, and the mean was the highest. "Most accurate on at least five of eight segments" held for the half Brier score (five) and not for the log-loss (four). The step $\eta_2 = \frac{1}{2}$ with tempered weights, the method of this paper, was added to the analysis after the baselines of three segments were visible and before any result of a two-level rule was computed; it was not part of the registered hypotheses. It gives five of eight segments and the highest mean.

Fourth held-out set (six segments: the next 200,000 rows each of Airlines, PokerHand and CoverType; method of Section 4 with fixed segmentation frozen). Registered hypotheses: on every segment not more than 0.5 points below the better of FIFO and reset; above all four in-context baselines on at least four of six segments, with the highest mean; on every segment not more than 0.3 points below FIFO; above the variant without the pool on at least four of six segments. All held (five of six segments; worst difference to FIFO -0.01 points; six of six against the variant without the pool).

Second backbone and second seed. With the method of Section 4 with fixed segmentation frozen and the same four hypotheses ("at least three of four" for sets of four segments): TabICL on four segments of the third held-out set and on the six segments of the fourth, all held; a second seed of TabPFN on the same four segments, three held and "most accurate on at least three of four segments" did not (two); a second seed on the six segments of the fourth held-out set, all held.

Grid, seeds 20 and 21 (half Brier score, step 10). Mean difference to the fast level alone at least -0.5 points (-0.14), above reset on all eight streams with 30 or 100 centroids, and within one point of reset with 5 centroids: all held. The step $\frac{1}{2}$ is 0.23 points lower on average and satisfies the same three statements.

Detected segmentation. Segmentation at detected changes replaced fixed segmentation after the boundary tests above; the merge rule, the experts and both levels are unchanged. Its constants ($H_{\mathrm{cp}} = 3$, $R_{\mathrm{cp}} = 5$, $\delta_{\mathrm{cp}} = 10^{-3}$, minimum segment of 300 rows, the 300-row window as signal) were set on development streams of the generator's development seeds 0 and 1: twenty streams with boundaries on batch edges (blocks of 500 and 2 000 rows, aligned and shifted, and variable blocks) and twelve with boundaries inside batches (offsets of 250, 1 170 and 37 rows). Because the window's errors do not depend on the pool, the segmentation was simulated offline from cached window predictions, and the online runs reproduced the simulated cuts exactly. A variant that leaves out the batch on each side of a declared change, which makes the segments pure, was less accurate on 500-row development blocks with 30 centroids (gain over reset -0.50 and -1.24 points against $+1.76$ and $+2.09$ without it) because it stores fewer rows; it was not used. The test streams, the criteria and the evaluator were committed before any test stream was run, and every test stream was run once (runs interrupted by GPU faults were resumed from their checkpoints). Criteria: (C1, primary) in each of the three length regimes, mean gain over reset at least $+1.0$ points, above reset on at least 18 of 24 streams, and a hierarchical-bootstrap 95% lower bound above zero; (C2) on the shifted grid, mean gain over fixed segmentation at least $+2.0$ points; (C3) on the eight aligned grid streams with 30 or 100 centroids, at least -0.3 points against fixed segmentation; (C4) on the 29 real streams, at least -0.1 points against FIFO and -0.2 against fixed segmentation; (C5) no more TFM calls and stored experts per batch than the archive with the same segments, and a mean difference to it of at least -0.3 points. Purity rule: the stored contexts meet the single-concept premise if at least 90% of the closes hold at least 95% rows of one concept, its majority form if at least 95% hold more than half, and otherwise the recall theorem is conditional on purity. All five criteria held; the purity rule gave its majority form (88% and 100%), that is, the strict form failed and the recall theorem remains conditional on pure contexts; the confirmation streams gave 87% and 100%. The registration described the test seeds 40 to 43 as unused; seeds 40 and 41 had in fact been used, two days earlier, for the registered oracle test of Table 3, with the same concept draws and, for fixed block lengths, the same rows after the offset. That test involved no detector and set no constant. We found the reuse after the evaluation, kept the registered verdicts as computed, and report the split by seeds as a post-hoc analysis (Section 5.2). We then registered a confirmation on seeds 50 to 53, unused by any test, with the methods, construction and criterion C1 unchanged, before any run; C1 held in all three regimes (Section 5.2). The resource and snapshot comparison of Table 5 was added in response to review and registered before it was run, with its policies, streams, statistics and reporting rule: the snapshot size k was fixed from the prefix timings alone, before any accuracy of the snapshot runs was read, as the larger of 2 and 4 within the time of MICE on all three prefixes, and 2 if neither was (the case that occurred); if the snapshot pool of that size had been within $+0.1$ points of MICE on average in any group, the abstract and conclusion would name the group (none was). The snapshot runs used backbone seed 0 and the existing runs of MICE on the same streams; the benchmark ran each configuration in a fresh process alone on the GPU, and no run failed.

.6 Controlled Grid and Learning Curves

Generator. A stream has $d = 10$ features, four nominal classes and three concepts. For $m \in \{5, 30, 100\}$ centroids, $C_j \sim \mathcal{U}([0,1]^{10})$ i.i.d. and mixture weights $w_j = u_j / \sum_i u_i$ with $u_j \sim \mathcal{U}(0,1)$. Each concept k has its own class vector $\lambda_k(j) \sim \mathcal{U}\{0,1,2,3\}$, drawn i.i.d. over centroids and independently across concepts: it is not a permutation, and two concepts may agree on a centroid (on our seeds the weight of such centroids is 18% to 48% averaged over the three pairs of concepts, 3% to 76% for single pairs), and with five centroids a concept can lack some classes (two to four present); there is no rejection or coverage constraint, and class priors follow from w and λ_k. A row of concept k draws $j \sim \mathrm{Categorical}(w)$, $\mathbf{x} = C_j + \mathcal{N}(0, 0.15^2 I_{10})$ and $y = \lambda_k(j)$. Concepts appear in the order $1, 2, 3$ in blocks of $L \in \{500, 2000\}$ rows, three cycles, so P_X is the same for all concepts. One NumPy generator per data seed draws C, w and λ first and then the rows. Data

TABLE 15
MICE versus FIFO with 3000 rows (fixed beforehand from timings; about 2.5 times cheaper than MICE on the three benchmark prefixes, cost not measured on these streams) on the 29 real streams: accuracy (%), difference (points) and paired two-sided 95% moving-block bootstrap interval (blocks of 20 batches; aggregate over independently resampled streams).

Stream	batches	MICE	FIFO 3000	Δ	95% CI
Electricity	452	81.22	81.53	-0.31	[-0.93, +0.43]
INSECTS abr.	527	77.85	72.94	+4.91	[+1.63, +7.47]
INSECTS grad.	240	80.42	77.08	+3.34	[+0.32, +6.24]
INSECTS inc.	569	67.77	68.15	-0.38	[-0.89, +0.13]
INSECTS inc.-rec.	798	80.80	75.32	+5.49	[+3.32, +8.17]
CoverType	1499	95.05	94.69	+0.36	[+0.05, +0.79]
INSECTS abr.-imb.	1499	79.81	79.51	+0.30	[-0.54, +1.39]
INSECTS grad.-imb.	1432	81.89	82.01	-0.11	[-0.58, +0.51]
INSECTS inc.-abr.	798	78.56	73.38	+5.17	[+3.52, +6.89]
Airlines	999	69.06	69.83	-0.77	[-1.06, -0.51]
Phishing	109	94.60	96.05	-1.45	[-2.23, -0.98]
PokerHand	999	92.97	95.66	-2.70	[-3.69, -1.87]
Rialto	821	82.19	75.25	+6.94	[+5.90, +8.12]
Spam	61	94.25	94.72	-0.48	[-1.10, +0.84]
Weather	180	81.07	81.61	-0.54	[-0.97, -0.24]
Airlines 3b	999	70.48	71.36	-0.88	[-1.12, -0.63]
Airlines 3c	999	65.23	66.25	-1.03	[-1.29, -0.75]
CoverType 3b	999	96.58	96.59	-0.01	[-0.11, +0.08]
CoverType 3c	999	95.72	96.04	-0.32	[-0.42, -0.19]
INSECTS 3b	999	81.67	81.97	-0.30	[-0.77, +0.31]
INSECTS 3c	999	85.66	85.72	-0.06	[-0.46, +0.42]
PokerHand 3b	999	93.98	93.89	+0.08	[-0.56, +0.68]
PokerHand 3c	999	92.16	94.30	-2.14	[-3.35, -1.13]
Airlines 4d	999	65.99	67.26	-1.27	[-1.55, -1.02]
Airlines 4e	999	65.59	67.12	-1.53	[-1.91, -1.21]
CoverType 4d	999	96.24	96.53	-0.29	[-0.38, -0.20]
CoverType 4e	999	97.19	97.31	-0.12	[-0.20, -0.04]
PokerHand 4d	999	94.49	95.49	-1.00	[-1.59, -0.44]
PokerHand 4e	999	92.34	94.33	-1.99	[-3.00, -0.92]
Mean		83.82	83.51	+0.31	[+0.09, +0.49]

TABLE 16
Controlled grid: parameters.

Features, classes, concepts	$d = 10, 4, 3$
Centroids m	5, 30, 100; $C_j \sim \mathcal{U}([0,1]^{10})$
Weights	$u_j \sim \mathcal{U}(0,1)$, normalised
Class of centroid j in concept k	$\lambda_k(j) \sim \mathcal{U}\{0,\dots,3\}$, independent
Noise	isotropic Gaussian, s.d. 0.15
Block length L, cycles	500 or 2000 rows, 3
Batch size, M	100, 1000 rows
Segments	300–700 rows (detected); 500 rows (fixed)

seeds: 0 and 1 for development (also of the change detector), 10 and 11 for the registered test, 20 and 21 for confirmation, 30 and 31 for the misaligned schedules, 40 and 41 for the oracle test, 40 to 43 for the test of detected segmentation, and 50 to 53 for its confirmation; the shifted streams reuse seeds 20 and 21 and draw their prefix from a separate generator, and the streams of Table 8 prepend rows of the first concept from a separate generator and also use blocks of lengths drawn uniformly from 500 to 2 500 rows.

Learning curves. Measured before the grid test, for data seeds 10 and 11: for each m and concept, one block of 4000 rows of that concept from the generator above. Rows 1–3000 form the context pool and rows 3001–4000 the 1000 test rows, disjoint. For $n \in \{50, 100, 200, 500, 1000, 2000\}$, three contexts are drawn without replacement from the pool (generator seeds 0, 1, 2), so contexts of different sizes and repetitions overlap and the same 1000 test rows are used throughout; the frozen TabPFN v2 (four estimators, random state equal to the data seed) predicts the test rows. The curve $a(n)$ is the mean accuracy over 2 seeds, 3 concepts and 3 repetitions (18 measurements per n, not 18 independent concept draws), interpolated linearly in $\log n$.

From curves to predictions. The averaged curve is inserted into the area term of Equation (7); visits 2 and 3 of a concept recur and enter with weight $\frac{1}{3}$ each (the first visit has no stored expert and contributes zero). The registered predictions used M stored rows on every recurrence; Section 5.1 also reports the form with $n_c = \min(M, (c-1)L)$. The observed gain of a cell is the mean over its two streams, and the Spearman correlations are over the six cells.

Shared randomness. The curve data of seeds 10 and 11 were generated by re-initialising the generator of the same seed with one 4000-row block per concept. They therefore share the concept definitions with the evaluated seed-10/11 streams, the first block's centroid draws coincide, and some rows are bit-identical feature vectors (0 to 1887 per stream), in runs where the two draw sequences of the same seed realign at a constant offset. The seed-20/21 streams share nothing with the curves. In a sensitivity analysis registered after this was found, we remeasured the curves on rows drawn, given the same concept definitions, from one independent NumPy generator per seed, centroid count and concept (seeded with $(20\,000+\text{seed}, m, k)$; no row in common with any evaluated stream). These counts and checks are produced by code/grid_overlap_check.py. The curves change by at most 0.015 at any n, the predictions by at most 0.6 points (100 centroids, $L = 500$: 9.9 instead of 10.4 registered, 8.9 instead of 9.2 with n_c), and the rank correlations are unchanged (0.94 on seeds 10 and 11, 0.83 on seeds 20 and 21). Stream evaluation uses backbone random state 0.

Streams. The grid and generator streams are produced by code/grid_streams.py and code/streams.py; generator streams have blocks of 2000 rows and three cycles over three or four concepts. Real streams with more than 100,000 rows are cut into segments of 100,000 (CoverType and one INSECTS stream: 150,000 for the first segment). The second held-out set consists of all streams of a public collection of drift data sets that have at least 5000 rows, at most ten classes and had not been used before. Baselines. LTM follows Algorithm 1 of Lourenço et al. [10] with a short-term share of 0.75. MOOE follows the original algorithm with linear experts on random Fourier features, generalised to several classes; it was tuned on the five development streams over feature type, interval length, step size and regularisation (72 configurations), and its grid was extended twice because the optimum lay on the boundary; the best configuration of every round is evaluated. The incremental learners use the default settings of river [22] with ten base models and the same batch protocol. Regret under adversarial sequences. With synthetic expert predictions and labels chosen to hurt the mixture or the current leader ($K \in \{2, 5\}$, $\gamma \in \{0.5, 0.9, 0.99, 1\}$, 400 rounds), the discounted regret at $\eta_2 = \frac{1}{2}$ stays below $\ln K/\eta_2$ in every configuration (at most 0.67 for $K = 2$), exceeds it at $\eta_2 = 2$ for $\gamma \geq 0.9$, and reaches 4.4 at $\eta_2 = 10$, $\gamma = 0.9$.

TABLE 17
Real streams: accuracy (%) per stream. Fast only: the fast level without the slow level; $\eta_2 = 10$: the pre-registered step; Trained: the best trained stream learner per stream; Δ: MICE minus FIFO in points.

Stream	FIFO	LTM	Reset	Win. ens.	Fast only	$\eta_2 = 10$	MICE	Trained	Δ
development									
elec2	81.0	80.8	79.9	80.7	80.6	81.2	81.2	80.1	+0.27
ins-abrupt-bal	76.5	76.3	76.5	77.4	78.0	77.9	77.9	76.7	+1.32
ins-gradual-bal	78.8	79.2	79.9	79.9	80.7	80.6	80.4	79.3	+1.59
ins-incr-bal	67.5	67.4	67.5	66.8	67.8	67.8	67.8	65.5	+0.30
ins-incr-reoc-bal	79.1	78.7	80.1	80.6	81.0	80.9	80.8	79.2	+1.65
mean	76.6	76.5	76.8	77.1	77.6	77.7	77.6	76.1	+1.03
first held-out set									
covertype	94.8	94.7	94.3	94.9	94.5	95.0	95.1	91.9	+0.27
ins-abrupt-imb	79.4	79.0	79.6	79.2	79.8	79.8	79.8	79.3	+0.41
ins-gradual-imb	81.6	81.3	81.7	81.4	81.9	81.9	81.9	81.4	+0.25
ins-incr-abrupt-bal	76.7	76.2	77.7	77.9	78.8	78.8	78.6	76.9	+1.86
mean	83.1	82.8	83.3	83.3	83.8	83.9	83.8	82.4	+0.70
second held-out set									
airlines	69.0	68.5	69.0	68.1	68.8	69.0	69.1	68.2	+0.05
phishing	94.6	94.6	94.6	94.1	94.6	94.6	94.6	94.4	+0.03
poker	92.6	89.9	84.4	91.1	90.2	92.5	93.0	82.8	+0.41
rialto	77.5	77.5	77.9	80.8	82.0	82.2	82.2	64.3	+4.73
spam	94.5	94.7	94.1	94.2	94.2	94.5	94.2	91.5	−0.25
weather	81.1	80.5	81.1	80.2	80.7	81.1	81.1	79.3	−0.01
mean	84.9	84.3	83.5	84.8	85.1	85.7	85.7	80.1	+0.83
third held-out set									
airlines-b	70.6	70.0	70.5	69.9	70.2	70.5	70.5	70.0	−0.08
airlines-c	65.1	64.5	65.0	64.3	65.0	65.2	65.2	65.0	+0.11
covertype-b	96.6	96.6	95.7	96.6	96.4	96.6	96.6	91.9	−0.03
covertype-c	95.7	95.8	91.9	95.7	95.7	95.7	95.7	80.2	+0.02
ins-b	81.4	81.0	81.4	81.2	81.7	81.7	81.7	81.1	+0.25
ins-c	85.4	85.0	85.5	85.3	85.7	85.7	85.7	85.5	+0.23
poker-b	93.4	91.5	85.0	92.2	91.7	93.7	94.0	76.5	+0.58
poker-c	90.9	88.3	83.4	89.8	90.2	91.7	92.2	74.6	+1.29
mean	84.9	84.1	82.3	84.4	84.6	85.1	85.2	78.1	+0.30
fourth held-out set (confirmatory)									
airlines-d	65.8	64.7	65.8	64.5	65.9	66.0	66.0	66.3	+0.18
airlines-e	65.5	64.8	65.3	64.2	65.5	65.6	65.6	65.0	+0.14
covertype-d	96.2	96.2	89.7	96.2	96.2	96.2	96.2	77.6	+0.05
covertype-e	97.2	97.2	96.7	97.2	97.1	97.2	97.2	92.9	−0.00
poker-d	94.1	91.6	84.6	92.9	92.1	94.1	94.5	77.9	+0.35
poker-e	91.8	89.1	83.2	90.5	89.9	92.4	92.3	75.8	+0.51
mean	85.1	83.9	80.9	84.2	84.5	85.2	85.3	75.9	+0.20
All streams	83.3	82.6	81.5	83.0	83.3	83.8	83.8		+0.57